

JavaScript — Standard Library

The built-in toolbox: every method on the objects you use daily — Array, String, Object, Map, JSON, Date, and the rest — taught with the common core in depth and the full surface mapped so nothing is invisible.

26 TOPICS · 10 PARTS · BUILT FOR LEARNING & PRINTING

How to read this

Start at the **Navigation Tree** — it maps the whole language; every name jumps to its section.

Each topic carries a tier badge. **CORE** topics are taught in depth, connected to real web & data-app work — learn these cold. **REFERENCE** topics are mapped lightly: enough to recognize and look up. Inside a topic, exhaustive lists (every operator, every loop form) are always shown so nothing stays hidden.

Code is syntax-highlighted. Look for **When to use** notes — they explain *why* you'd reach for a construct and which alternative to pick. **Version** notes flag when a feature landed (ES2020+). A **API Index** at the end lists every operator and keyword.

Print or save as PDF with **Ctrl/Cmd + P** — the layout flows continuously and is print-tuned.

Navigation Tree

The whole language at a glance. teal = core (learn deeply) · grey = reference (know it exists). Every name links to its section.

```
├─ Array
│   ├── Creating & checking arrays
│   ├── Adding, removing & reordering
│   ├── Finding & testing
│   ├── Transforming – map · filter · reduce
│   ├── Iterating & joining
│   └─ Copying & immutable updates
├─ String
│   ├── Inspecting & searching
│   ├── Extracting & slicing
│   ├── Transforming & cleaning
│   └─ Splitting, joining & padding
├─ Number & Math
│   ├── Numbers – parsing, formatting, checks
│   └─ Math – rounding, ranges, random
├─ Object
│   ├── Reading, building & merging
│   └─ Protecting & inspecting
├─ Keyed Collections
│   ├── Map
│   └─ Set
```

- | └ WeakMap & WeakSet
- | └ JSON
- | └ └ JSON.parse & JSON.stringify
- | └ Dates & Internationalization
- | └ └ Date
- | └ └ Intl – formatting for humans
- | └ RegExp
- | └ └ Regular expressions
- | └ Promise (static methods)
- | └ └ Promise combinators
- | └ More built-ins
 - | └ └ Symbol
 - | └ └ Iterator helpers
 - | └ └ Error types
 - | └ └ Binary data – typed arrays (brief)

Part 1 • Array

Arrays are ordered lists — the structure you reach for whenever you have “many of something”; rows from a database, items in a cart, results from an API. This part covers their full method surface, grouped by what you’re trying to do.

Creating & checking arrays CORE

IN PLAIN TERMS

An array is an ordered list of values, written with square brackets: `[1, 2, 3]`. Each value sits at a numbered position called an **index**, and counting starts at **0** — so the first item is at index `0`, the second at `1`, and so on. Before you work with something, you often need to *make* an array (from scratch, or from some other data) or *check* that what you have really is one. This section covers both.

You’ll create arrays three main ways: the literal `[]` (by far the most common), `Array.from()` to build one from something iterable or array-like, and `Array.of()` for a quick list of known values. To verify you actually have an array — which matters because `typeof [] === "object"`, unhelpfully — the one reliable test is `Array.isArray()`.

▼ KEY TERMS IN THIS SECTION

<code>===</code>	Strict equality operator — compares two values and returns <code>true</code> only if they’re the same type and value, with no automatic conversion. Your default for comparing.
<code>==</code>	Loose equality operator — compares two values <i>after</i> converting them to a common type, which causes surprises. Avoid in favour of <code>===</code> .
<code>=></code>	Arrow — defines an <i>arrow function</i> , a short way to write a function (e.g. <code>x => x * 2</code>). The arrow separates the inputs from what the function does.
<code>**</code>	Exponentiation operator — raises a number to a power (<code>2 ** 3</code> is <code>8</code>).
array-like	Array-like — an object with a <code>length</code> and numbered keys but no array methods (e.g. a DOM <code>NodeList</code>); convert with <code>Array.from</code> .
iterable	Iterable — any value you can step through one item at a time with <code>for...of</code> (arrays, strings, Maps, Sets).
promise	Promise — an object standing in for a value that isn’t ready yet; it later succeeds or fails.
element	Element — a node created from an HTML tag (a <code><div></code> , <code><p></code> , <code><button></code>); the most common kind of node.
index	Index — an item’s numbered position in an array or string, starting at 0.
DOM	DOM (Document Object Model) — the browser’s live, in-memory tree of objects representing the page; changing it changes what’s on screen.

`Array.from` with a *map function* is a clean way to generate a sequence — handy for building ranges or placeholder rows.

```
const nums = [1, 2, 3];           // array literal – the everyday way
const empty = [];                 // empty, ready to fill
const rows = Array.from(nodeList); // make a real array from array-like/iterable
const chars = Array.from("hi");   // ["h", "i"]
const mapped = Array.from({length: 3}, (_, i) => i * 2); // [0, 2, 4]
const few = Array.of(7);          // [7] (Array(7) would make 7 empty slots!)

Array.isArray(nums);              // true – the reliable check
typeof nums;                      // "object" – why isArray exists
nums.length;                     // 3 – count of items
```

GOTCHA `new Array(3)` creates an array with **3 empty slots**, not `[3]` — a classic trap. Use `Array.of(3)` for a single-element array, and the literal `[]` for everything else. Empty slots ("holes") also behave oddly with some methods, so avoid them.

WHEN TO USE Arrays exist to hold an *ordered* collection where position matters and duplicates are fine — a feed of posts, a queue of jobs, the rows of a table. When you instead need fast lookup by a unique key, or guaranteed-unique values, reach for `Map` / `Set` (later in this book). Use `Array.from` specifically to convert the *array-like* things the browser hands you (a `NodeList` from the DOM, the `arguments` object) into real arrays so the full method toolbox below becomes available.

Creating & checking

`[]` literal

Array literal — the standard way to write a list: `[1, 2, 3]`. Use this unless you have a specific reason not to.

Array.from() `Array.from(src, mapFn?)`

Builds a new array from an iterable (string, Set, Map) or array-like (NodeList, `arguments`). Optional map function transforms each item as it's copied.

Array.of() `Array.of(...items)`

Makes an array from its arguments: `Array.of(7) → [7]`. Exists to avoid the `new Array(7)` empty-slots trap.

Array.fromAsync() `await Array.fromAsync(src)`

Like `Array.from` but for **async** iterables — awaits each value. Returns a promise. Useful for draining an async stream into an array. ES2025.

Array.isArray() `Array.isArray(x)`

The reliable "is this an array?" test. Always use this rather than `typeof` (which returns `"object"` for arrays).

new Array(n) constructor

Creates an array of `n` empty slots. Mostly a foot-gun; prefer literals or `Array.from({length:n})`.

.length `arr.length`

The number of items. Writable — setting it shorter truncates the array; a quick way to empty one is `arr.length = 0`.

Adding, removing & reordering CORE

IN PLAIN TERMS

Once you have an array, you'll constantly add items to it, take items out, and change their order. Two important words here: a method that **mutates** changes the original array in place; a method that returns a **new array** leaves the original untouched. Knowing which is which prevents a whole category of bugs, so each item below is labeled.

The mutating workhorses are `push` / `pop` (add/remove at the **end**) and `shift` / `unshift` (at the **start**), plus `splice` (insert/remove anywhere) and `sort` / `reverse` (reorder). They all change the array in place and are perfect when you *own* the array. When you must not mutate (shared state, UI data), use the immutable equivalents in the *Copying & immutable updates* section.

▼ KEY TERMS IN THIS SECTION

<code>=></code>	Arrow — defines an <i>arrow function</i> , a short way to write a function (e.g. <code>x => x * 2</code>). The arrow separates the inputs from what the function does.
<code>**</code>	Exponentiation operator — raises a number to a power (<code>2 ** 3</code> is <code>8</code>).
<code>comparator</code>	Comparator — a function you give <code>sort</code> that returns negative/zero/positive to decide the order of two items.
<code>immutable</code>	Immutable update — making a new value with the change instead of altering the original.
<code>argument</code>	Argument — the actual value you pass into a function when you call it.
<code>mutate</code>	Mutate — to change a value in place; a mutating method alters the original array/object.
<code>index</code>	Index — an item's numbered position in an array or string, starting at 0.

`push` / `pop` at the end are fastest; `shift` / `unshift` at the start are slower because every other item must shift position.

```
const stack = [];  
stack.push("a", "b");    // add to END → ["a","b"] (returns new length)  
stack.pop();            // remove from END → returns "b"  
  
const queue = ["x", "y"];  
queue.shift();          // remove from START → returns "x"  
queue.unshift("w");     // add to START → ["w","y"]  
  
const list = [1, 2, 3, 4, 5];  
list.splice(1, 2);      // remove 2 items from index 1 → list is [1,4,5]  
list.splice(1, 0, 9);   // insert 9 at index 1 (remove 0) → [1,9,4,5]  
  
[3, 1, 2].sort();        // [1,2,3] but BEWARE default is string order  
[3, 1, 2].sort((a, b) => a - b); // [1,2,3] numeric — pass a comparator  
["a","b"].reverse();    // ["b","a"] (mutates)
```

GOTCHA `.sort()` with no argument converts items to **strings** and sorts by character code — so `[10, 9, 100].sort()` gives `[10, 100, 9]`. For numbers, always pass a comparator: `.sort((a, b) => a - b)`. Note `sort` and `reverse` **mutate**; use `toSorted` / `toReversed` (immutable, ES2023) when you must not change the original.

WHEN TO USE These mutating methods exist because changing a list in place is efficient and natural when the array is *yours* — building up a result, managing a stack or queue. The danger is only when an array is **shared**: if another part of the app holds the same array (very common with UI state), mutating it changes their copy too, causing bugs that are hard to trace. Rule of thumb: mutate freely on arrays you just created locally; switch to the immutable versions for anything passed in or stored in shared state.

Add / remove (all mutate)

.push() `arr.push(...items)`

Adds to the **end**. Returns the new length. The most common way to grow an array.

.pop() `arr.pop()`

Removes and returns the **last** item. Pairs with `push` to use an array as a stack.

.unshift() `arr.unshift(...items)`

Adds to the **start**. Returns new length. Slower than `push` (shifts all items).

.shift() `arr.shift()`

Removes and returns the **first** item. Pairs with `push` to use an array as a queue.

.splice() `arr.splice(start, delCount, ...add)`

Swiss-army knife: removes `delCount` items at `start` and/or inserts new ones. Returns the removed items. Mutates.

.fill() `arr.fill(value, start?, end?)`

Overwrites a range with a fixed value. `new Array(3).fill(0) → [0,0,0]`.

.copyWithin() `arr.copyWithin(target, start, end?)`

Copies part of the array to another position within itself. Rare.

Reorder (mutate)

.sort() `arr.sort(compareFn?)`

Sorts in place. **Pass a comparator** for numbers/custom order; the default is string order. Stable since ES2019.

.reverse() `arr.reverse()`

Reverses order in place.

Finding & testing CORE

IN PLAIN TERMS

Often you don't want the whole list — you want *one* item out of it, or a yes/no answer about it. "Is there a user with this id?" "Does every field pass validation?" "Where is this item?" These methods answer those questions. Most take a small function (a **callback**) that you use to describe what you're looking for.

To get an item, use `find` (the item itself) or `findIndex` (its position). To get a yes/no, use `includes` (is this exact value present?), `some` (does *any* item pass?), or `every` (do *all* items pass?). These read almost like English and replace a lot of manual loops.

▼ KEY TERMS IN THIS SECTION

<code>===</code>	Strict equality operator — compares two values and returns <code>true</code> only if they're the same type and value, with no automatic conversion. Your default for comparing.
<code>==</code>	Loose equality operator — compares two values <i>after</i> converting them to a common type, which causes surprises. Avoid in favour of <code>===</code> .
<code>=></code>	Arrow — defines an <i>arrow function</i> , a short way to write a function (e.g. <code>x => x * 2</code>). The arrow separates the inputs from what the function does.
<code>...</code>	Spread/rest operator (three dots) — either <i>spreads</i> a collection out into its items, or <i>gathers</i> loose items into one collection, depending on where it's used.
<code>**</code>	Exponentiation operator — raises a number to a power (<code>2 ** 3</code> is <code>8</code>).
short-circuit	Short-circuit — stopping early once the answer is known (<code>some</code> stops at first match, <code>every</code> at first failure).
callback	Callback — a function you pass to a method (like <code>map</code> or <code>filter</code>) for it to call on each item.
index	Index — an item's numbered position in an array or string, starting at 0.
NaN	NaN ("Not a Number") — the result of an invalid math operation, like <code>0/0</code> .

`find` / `some` / `every` take a test function; `includes` / `indexOf` take a plain value. Reach for the function versions when matching on a property.

```
const users = [{id:1,active:true},{id:2,active:false},{id:3,active:true}];

users.find(u => u.id === 2);           // {id:2, active:false} - first match, or undefined
users.findIndex(u => u.id === 2);       // 1 - position, or -1 if none
users.findLast(u => u.active);          // {id:3,...} - search from the END (ES2023)

[1, 2, 3].includes(2);                 // true - exact value present? (uses ===)
users.some(u => !u.active);             // true - is ANY user inactive?
users.every(u => u.active);             // false - are ALL users active?

[1, 2, 3].indexOf(2);                  // 1 - position of an exact value (or -1)
```

GOTCHA `includes` and `indexOf` compare with strict equality, so they find primitives but **not** equivalent objects:

`[{id:1}].includes({id:1})` is `false` (different object references). To find an object by its contents, use `find` with a test: `arr.find(o => o.id === 1)`. Also, `indexOf(NaN)` never finds `NaN` — `includes(NaN)` does.

WHEN TO USE These exist to make intent obvious and to **short-circuit** — `some` stops at the first match, `every` stops at the first failure, so they're efficient as well as readable. Before them, every "does any row...?" check was a hand-written loop with a flag variable; `some` / `every` say the same thing in one line. Use `find` over `filter()[0]` when you only want one result — `filter` always scans the whole array and builds a new one, while `find` stops as soon as it hits a match.

Get an item / position

.find() `arr.find(testFn)`

Returns the **first** item for which the test returns true, or `undefined`. The go-to for "get the record where...".

.findIndex() `arr.findIndex(testFn)`

Like `find` but returns the **index** (or `-1`). Use when you need the position, e.g. to replace or remove.

.findLast() `arr.findLast(testFn)`

Searches from the **end**. Returns the last match. **ES2023**.

.findLastIndex() `arr.findLastIndex(testFn)`

Index version of `findLast`. **ES2023**.

.indexOf() `arr.indexOf(value, from?)`

First index of an exact value (`===`), or `-1`. For values, not tests.

.lastIndexOf() `arr.lastIndexOf(value)`

Last index of an exact value.

.at() `arr.at(index)`

Item at an index, with **negative** support: `arr.at(-1)` is the last item — cleaner than `arr[arr.length-1]`. **ES2022**.

Yes / no questions

.includes() `arr.includes(value)`

`true` if an exact value is present. Correctly finds `NaN`. For primitives.

.some() `arr.some(testFn)`

`true` if **at least one** item passes the test. Short-circuits on first success.

.every() `arr.every(testFn)`

`true` if **all** items pass (and `true` for an empty array). Short-circuits on first failure.

Transforming arrays CORE

IN PLAIN TERMS

This is the heart of working with data in JavaScript. Instead of writing loops, you *transform* one array into another with three core methods. `map` makes a new array by changing every item ("give me just the names"). `filter` makes a new array keeping only items that pass a test ("only the active users"). `reduce` boils the whole array down to a single value ("the total price"). They don't change the original — each returns a fresh array (or value).

`map`, `filter`, and `reduce` are the backbone of data work — rendering lists in UIs, shaping API responses, computing summaries. They **chain**: `data.filter(...).map(...)` reads as a pipeline of steps. Because they return new arrays and never mutate, they fit perfectly with state-driven UIs. Learn these three cold; the rest of this book leans on them.

▼ KEY TERMS IN THIS SECTION

<code>=></code>	Arrow — defines an <i>arrow function</i> , a short way to write a function (e.g. <code>x => x * 2</code>). The arrow separates the inputs from what the function does.
<code>...</code>	Spread/rest operator (three dots) — either <i>spreads</i> a collection out into its items, or <i>gathers</i> loose items into one collection, depending on where it's used.
<code>**</code>	Exponentiation operator — raises a number to a power (<code>2 ** 3</code> is <code>8</code>).
<code>accumulator</code>	Accumulator — in <code>reduce</code> , the running result carried from one item to the next.
<code>declarative</code>	Declarative — describing <i>what</i> the UI should look like for given data and letting the framework update the DOM (vs. imperative step-by-step DOM calls).
<code>element</code>	Element — a node created from an HTML tag (a <code><div></code> , <code><p></code> , <code><button></code>); the most common kind of node.
<code>mutate</code>	Mutate — to change a value in place; a mutating method alters the original array/object.
<code>index</code>	Index — an item's numbered position in an array or string, starting at 0.
<code>rest</code>	Rest — using <code>...</code> to collect several values into a single array or object.

`map` and `filter` each return a new array, so they chain. `reduce` is the most general — `map` and `filter` can both be written as a `reduce`, but the named ones read better.

```
const products = [
  {name: "Pen",    price: 2, inStock: true},
  {name: "Desk",  price: 200, inStock: false},
  {name: "Lamp",  price: 40, inStock: true},
];

// map - transform every item into something new
const names = products.map(p => p.name);           // ["Pen", "Desk", "Lamp"]
const withTax = products.map(p => ({...p, price: p.price * 1.1}));

// filter - keep only items that pass the test
const available = products.filter(p => p.inStock); // Pen & Lamp

// reduce - collapse to a single value (sum, object, anything)
const total = products.reduce((sum, p) => sum + p.price, 0); // 242
// the 0 is the STARTING value (the "accumulator")

// chain them into a pipeline:
const inStockNames = products
  .filter(p => p.inStock)
  .map(p => p.name);           // ["Pen", "Lamp"]
```

flatMap & flat — when items expand into lists

`flatMap` is `map` followed by a one-level `flat` — ideal when each item produces several results you want in one flat list.

```
const orders = [{items: ["a", "b"]}, {items: ["c"]}];
orders.flatMap(o => o.items);           // ["a", "b", "c"] - map then flatten one level
[1, [2, [3]]].flat();                  // [1, 2, [3]]   - flatten one level
[1, [2, [3]]].flat(Infinity);          // [1, 2, 3]     - flatten all levels
```

GOTCHA `reduce` without a starting value uses the first element as the initial accumulator and can throw on an empty array — always pass the initial value (`, 0` or `, {}` or `, []`). Also, inside `map` / `filter` callbacks, returning nothing gives `undefined` — for `map`, remember to actually `return` the new value (or use a concise arrow without braces).

WHEN TO USE These exist to make data transformation **declarative** — you say *what* you want, not the mechanics of looping and pushing. The payoff is threefold: the code reads as a description of the result, there's no manual index bookkeeping to get wrong, and because nothing is mutated, the same input always gives the same output (easy to reason about and test). This is *why* modern JavaScript favors them over `for` loops for shaping data: a `.filter().map()` pipeline is the standard way data flows through a UI. Drop back to a real loop only when you need to `break` early or `await` each step.

The core three

.map() `arr.map(fn)`

Returns a **new** array with `fn` applied to each item. "Transform every element." Same length as the original.

.filter() `arr.filter(testFn)`

Returns a **new** array of only the items that pass the test. "Keep the ones that match."

.reduce() `arr.reduce(fn, initial)`

Collapses the array to a single value by accumulating. `fn(accumulator, item)` runs per element. Always pass `initial`. The most general transform.

`.reduceRight()` `arr.reduceRight(fn, initial)`

Like `reduce` but processes right-to-left. Niche.

`.flatMap()` `arr.flatMap(fn)`

`map` then flatten one level — for when each item yields several results.

`.flat()` `arr.flat(depth?)`

Flattens nested arrays by `depth` levels (default 1). `flat(Infinity)` fully flattens. ES2019.

Iterating & joining CORE

IN PLAIN TERMS

Sometimes you don't want to build a new array — you just want to *do something* with each item (log it, render it, send it), or turn the list into a single string. These methods walk the array for those purposes.

`forEach` runs a function for every item purely for its side-effects (it returns nothing). `join` turns an array into a string with a separator. `keys` / `values` / `entries` give you iterators for use in `for...of` loops, with `entries` handing you index-and-value pairs.

▼ KEY TERMS IN THIS SECTION

- `=>` Arrow — defines an *arrow function*, a short way to write a function (e.g. `x => x * 2`). The arrow separates the inputs from what the function does.
- `**` Exponentiation operator — raises a number to a power (`2 ** 3` is `8`).
- `side-effect` Side-effect — a change to something outside the function (logging, updating the DOM) rather than returning a value.
- `iterator` Iterator — an object that produces values one at a time via `.next()`; what `keys` / `values` / `entries` return.
- `element` Element — a node created from an HTML tag (a `<div>`, `<p>`, `<button>`); the most common kind of node.
- `render` Render — to produce or update what the user sees on screen from data.
- `index` Index — an item's numbered position in an array or string, starting at 0.

`join` is the clean way to build text from a list — far better than string concatenation in a loop.

```
["a","b","c"].forEach((item, i) => console.log(i, item)); // side-effects only
```

```
["a","b","c"].join(", "); // "a, b, c" - array → string
```

```
["a","b","c"].join(""); // "abc"
```

```
[1,2,3].join(" / "); // "1 / 2 / 3"
```

```
for (const [i, val] of ["x","y"].entries()) { // index + value together
  console.log(i, val); // 0 "x", then 1 "y"
}
```

GOTCHA `forEach` cannot be stopped early (no `break`) and does not wait for `await` inside it — if you need either, use a `for...of` loop instead. Also, `forEach` returns `undefined`, so it can't be chained like `map` / `filter`. If you find yourself building a result inside `forEach`, you probably want `map`, `filter`, or `reduce`.

WHEN TO USE `forEach` exists for the case where you genuinely only want a side-effect for each item and aren't producing a new array — though in practice a `for...of` loop is often clearer and more capable (it supports `break` and `await`). `join` exists because building strings from lists is so common, and doing it with `+=` in a loop is both slower and more error-prone (stray separators). Reach for `join` any time you're turning a list into displayable or storable text — CSV rows, comma lists, URL query parts.

Iterate / stringify

.forEach() `arr.forEach(fn)`

Runs `fn(item, index)` for each element, for side-effects. Returns `undefined`. Can't `break` or `await`.

.join() `arr.join(separator?)`

Joins items into a string with `separator` (default `,`). `null` / `undefined` become empty strings.

.entries() `arr.entries()`

Iterator of `[index, value]` pairs — use with `for...of` when you need both.

.keys() `arr.keys()`

Iterator of indexes.

.values() `arr.values()`

Iterator of values (what `for...of` uses by default). The entry point to Iterator helpers (later).

.toString() `arr.toString()`

Like `join(",")`; called automatically when an array is coerced to text.

Copying & immutable updates CORE

IN PLAIN TERMS

A reminder from Book 1: arrays are shared by **reference**, so handing your array to another function lets it change your copy. The safe pattern is to make a *new* array with the change rather than altering the original — this is called an **immutable update**, and it's how state-driven apps (like React) avoid a whole class of bugs. This section gathers the tools for copying and for the newer "change but return a copy" methods.

For copies: spread `[...arr]` and `slice()` make shallow copies; `concat` joins arrays into a new one. For immutable *edits*, ES2023 added `toSorted`, `toReversed`, `toSpliced`, and `with` — the copying twins of `sort`, `reverse`, `splice`, and index assignment. They do the same job without touching the original.

▼ KEY TERMS IN THIS SECTION

<code>!==</code>	Strict inequality operator — the opposite of <code>===</code> ; <code>true</code> when two values are <i>not</i> strictly equal.
<code>==</code>	Loose equality operator — compares two values <i>after</i> converting them to a common type, which causes surprises. Avoid in favour of <code>===</code> .
<code>=></code>	Arrow — defines an <i>arrow function</i> , a short way to write a function (e.g. <code>x => x * 2</code>). The arrow separates the inputs from what the function does.
<code>...</code>	Spread/rest operator (three dots) — either <i>spreads</i> a collection out into its items, or <i>gathers</i> loose items into one collection, depending on where it's used.
<code>**</code>	Exponentiation operator — raises a number to a power (<code>2 ** 3</code> is <code>8</code>).
reference	Reference — a pointer to an object; copying it copies the pointer, so two names can share one object.
deep copy	Deep copy — a fully independent duplicate, including nested objects (<code>structuredClone</code>).
shallow	Shallow copy — a copy one level deep; nested objects are still shared with the original.
mutate	Mutate — to change a value in place; a mutating method alters the original array/object.
spread	Spread — using <code>...</code> to expand a list or object into its individual pieces.
index	Index — an item's numbered position in an array or string, starting at 0.
node	Node — any single point in the DOM tree: an element, a piece of text, a comment, or the document itself.

The `to...` methods and `with` are the immutable versions of the mutating ones — same operation, fresh array. Perfect for updating state without side-effects.

```
const original = [3, 1, 2];

// shallow copies
const copy = [...original];           // spread — the usual way
const copy2 = original.slice();       // slice with no args — same effect
const joined = original.concat([4]); // new array [3,1,2,4]

// immutable edits (ES2023) — return a NEW array, original untouched:
const sorted = original.toSorted((a,b) => a-b); // [1,2,3]; original still [3,1,2]
const reversed = original.toReversed();        // [2,1,3]
const replaced = original.with(0, 99);         // [99,1,2] (index 0 → 99)
const spliced = original.toSpliced(1, 1);      // [3,2] (remove index 1)

// immutable "append" / "remove" with spread + filter:
const added = [...original, 4];
const removed = original.filter(x => x !== 1);
```

GOTCHA All these copies are **shallow** — the new array is independent, but objects *inside* it are still shared with the original. Changing `copy[0].name` also changes `original[0].name`. For a fully independent deep copy, use `structuredClone(arr)`.

WHEN TO USE These exist to make "change without mutating" easy. Before ES2023, sorting a prop without wrecking the original meant `[...arr].sort()` — copy first, then mutate the copy. `toSorted` and friends express that intent directly and avoid the easy mistake of forgetting the copy. **Why it matters so much in UI code:** frameworks detect changes by comparing whether the array *reference* changed; if you mutate in place the reference is the same and the UI may not update, but if you produce a new array it re-renders correctly. So immutable updates aren't just tidy — they're often required for the screen to refresh.

VERSION `toSorted`, `toReversed`, `toSpliced`, and `with` are all **ES2023**. `structuredClone` is a relatively recent global (Node 17+, all modern browsers) for true deep copies.

Copy / combine

`[...arr]` spread

Shallow copy via spread — the everyday way to duplicate before mutating, or to combine: `[...a, ...b]`.

`.slice()` `arr.slice(start?, end?)`

Returns a shallow copy of a portion (or all, with no args). Does **not** mutate (unlike `splice`).

`.concat()` `arr.concat(...arrays)`

Returns a new array joining this one with others/values. Non-mutating.

`structuredClone()` `structuredClone(arr)`

Deep copy — duplicates nested objects too. Use when a shallow copy isn't enough.

Immutable edits (ES2023)

`.with()` `arr.with(index, value)`

Copy with one index replaced. The immutable form of `arr[i] = value`.

`.toSorted()` `arr.toSorted(cmp?)`

Sorted copy — like `sort` but leaves the original alone.

.toReversed() `arr.toReversed()`

Reversed copy.

.toSpliced() `arr.toSpliced(start, delCount, ...add)`

Spliced copy — insert/remove returning a new array.

Part 2 • String

Text is everywhere in an app — names, messages, URLs, user input, IDs. Strings are how you hold and shape it. This part covers the full method surface for inspecting, slicing, cleaning, and splitting text.

Inspecting & searching strings CORE

IN PLAIN TERMS

A **string** is a piece of text, written in quotes: `"hello"`, `'hi'`, or ``hey``. Strings are **immutable** — every method that "changes" a string actually returns a *new* one and leaves the original alone. Before doing anything with text, you often need to ask questions about it: how long is it? does it contain this word? does it start with that prefix? These methods answer those.

The everyday checks are `includes` (does it contain this?), `startsWith` / `endsWith` (prefix/suffix tests — great for file extensions, URL checks), and `indexOf` (where is this substring?). `length` gives the character count. These read clearly and replace fiddly manual checks.

▼ KEY TERMS IN THIS SECTION

<code>...</code>	Spread/rest operator (three dots) — either <i>spreads</i> a collection out into its items, or <i>gathers</i> loose items into one collection, depending on where it's used.
<code>**</code>	Exponentiation operator — raises a number to a power (<code>2 ** 3</code> is <code>8</code>).
regular expression	Regular expression — a pattern for matching text, written between slashes: <code>/abc/</code> .
expression	Expression — any piece of code that produces a value, like <code>2 + 2</code> or <code>user.name</code> .
immutable	Immutable update — making a new value with the change instead of altering the original.
Unicode	Unicode — the standard that assigns a number to every character; JavaScript strings are sequences of UTF-16 code units.
index	Index — an item's numbered position in an array or string, starting at 0.
<code>includes</code> / <code>startsWith</code> / <code>endsWith</code> return booleans and are the most readable way to test text. <code>indexOf</code> gives a position when you need it.	

```
const email = "ada@example.com";
```

```
email.length;           // 15
email.includes("@");     // true  - contains substring?
email.endsWith(".com");  // true  - useful for extensions/domains
email.startsWith("ada"); // true
email.indexOf("@");      // 3    - position (or -1 if absent)
email.lastIndexOf(".");  // 11   - last occurrence

"Hello".charAt(0);       // "H"
"Hello"[0];              // "H"  - bracket access also works
"Hello".at(-1);          // "o"  - negative index (ES2022)
"café".codePointAt(3);   // 233  - the character's Unicode number
```

GOTCHA String searches are **case-sensitive**: `"Hello".includes("hello")` is `false`. Lower-case both sides first (`s.toLowerCase().includes(...)`) for case-insensitive checks. Also, `length` counts UTF-16 code units, so some emoji count as 2 — use `[...str].length` to count visible characters.

WHEN TO USE These exist so you can branch on the *content* of text without reaching for regular expressions (which are powerful but overkill for simple checks). `endsWith(".pdf")` is clearer and faster than a regex for a file-type check. Use `includes` for a plain "contains" test; step up to a regular expression (later in this book) only when the pattern is genuinely variable — digits, optional parts, repetition.

Length & character access

.length `str.length`

Number of UTF-16 code units. For most text this is the character count; some emoji count as 2.

.at() `str.at(i)`

Character at an index, with negative support (`str.at(-1)` is the last). **ES2022**.

.charAt() `str.charAt(i)`

Character at an index (older form; returns `""` past the end rather than `undefined`).

[i] `str[i]`

Bracket access to a character. Read-only (strings are immutable).

.charCodeAt() `str.charCodeAt(i)`

The UTF-16 code-unit number at an index.

.codePointAt() `str.codePointAt(i)`

The full Unicode code-point number — correct for characters beyond the basic range.

Searching & testing

.includes() `str.includes(sub)`

`true` if the substring is present. Case-sensitive.

.startsWith() `str.startsWith(sub)`

`true` if the string begins with the substring.

.endsWith() `str.endsWith(sub)`

`true` if it ends with the substring — handy for extensions, suffixes.

.indexOf() `str.indexOf(sub)`

First position of the substring, or `-1`.

.lastIndexOf() `str.lastIndexOf(sub)`

Last position of the substring, or `-1`.

.search() `str.search(regex)`

Like `indexOf` but takes a regular expression; returns position or `-1`.

.match() / **.matchAll()** `str.match(regex)`

Find regex matches (see Regular expressions). `matchAll` returns all matches with groups.

Extracting & slicing CORE

IN PLAIN TERMS

Often you want just *part* of a string — the first three letters, everything after the `@`, the file name without its path. **Slicing** means taking a section out by position. The main tool is `slice`, which takes a start and (optional) end position and returns that piece as a new string.

`slice(start, end)` is the one to learn — it's predictable and supports negative indexes (counting from the end). `substring` is similar but older and handles negatives differently (treats them as 0), so prefer `slice`. The original string is never changed.

▼ KEY TERMS IN THIS SECTION

**	Exponentiation operator — raises a number to a power (<code>2 ** 3</code> is <code>8</code>).
regular expression	Regular expression — a pattern for matching text, written between slashes: <code>/abc/</code> .
expression	Expression — any piece of code that produces a value, like <code>2 + 2</code> or <code>user.name</code> .
argument	Argument — the actual value you pass into a function when you call it.
index	Index — an item's numbered position in an array or string, starting at 0.

`slice` with negative numbers counts from the end — `slice(-4)` is "the last four characters," which is very handy.

```
const s = "ada@example.com";

s.slice(0, 3);           // "ada"   - from index 0 up to (not including) 3
s.slice(4);             // "example.com" - from index 4 to the end
s.slice(-4);            // ".com"   - last 4 characters (negative = from end)
s.slice(-4, -1);        // ".co"

const at = s.indexOf("@");
const user = s.slice(0, at); // "ada"   - everything before @
const domain = s.slice(at + 1); // "example.com" - everything after
```

GOTCHA `slice`, `substring`, and the deprecated `substr` are easy to confuse. Use `slice`: it's the most consistent and the only one with useful negative-index support. Avoid `substr` (it's legacy and takes a *length* as its second argument, not an end position).

WHEN TO USE Slicing exists for the constant task of pulling fields out of structured text — a username from an email, a year from a date string, an extension from a filename. It pairs naturally with `indexOf`: find the marker, then slice around it. For more complex extraction (variable patterns, multiple fields), a regular expression with capture groups is the next step up.

Extract a portion

.slice() `str.slice(start, end?)`

Returns the section from `start` up to (not including) `end`. Supports negative indexes. Your default.

.substring() `str.substring(start, end?)`

Like `slice` but treats negatives as `0` and swaps args if `start > end`. Prefer `slice`.

.substr() `str.substr(start, length)`

Deprecated. Second argument is a length, not an end. Don't use in new code.

Transforming & cleaning CORE

IN PLAIN TERMS

User input is messy — extra spaces, wrong capitalization, characters you need to swap out. These methods return a cleaned-up *new* string. The essentials: trim off surrounding whitespace, change case, and replace text.

`trim` removes leading/trailing whitespace (essential for form input). `toLowerCase` / `toUpperCase` normalize case (for comparisons and display). `replace` / `replaceAll` swap text out. `padStart` / `padEnd` add filler to a target width (great for formatting numbers and IDs). All return new strings.

▼ KEY TERMS IN THIS SECTION

- ...** Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.
- **** Exponentiation operator — raises a number to a power (`2 ** 3` is `8`).
- reference** Reference — a pointer to an object; copying it copies the pointer, so two names can share one object.
- immutable** Immutable update — making a new value with the change instead of altering the original.
- template** Template literal — a string written in backticks that supports `${ }` value slots and multiple lines.
- Unicode** Unicode — the standard that assigns a number to every character; JavaScript strings are sequences of UTF-16 code units.
- locale** Locale — a language/region setting (like `en-US`) that controls formatting of dates, numbers, and text.

Use `replaceAll` to swap every occurrence; plain `replace` with a string only changes the first. `trim` on form input prevents "why won't my password match" bugs from stray spaces.

```
" ada@example.com ".trim();           // "ada@example.com" - clean form input
"Hello".toLowerCase();               // "hello" - for case-insensitive compare
"Hello".toUpperCase();               // "HELLO"

"2024-01-05".replaceAll("-", "/");   // "2024/01/05" - replace every match
"a.b.c".replace(".", "_");           // "a_b.c" - replace() does only the FIRST

"5".padStart(3, "0");                // "005" - pad to width 3 (invoice numbers)
"42".padEnd(5, ".");                 // "42..."

" trim only one side ".trimStart();  // "trim only one side "
"café".normalize();                  // normalize Unicode accents for safe comparison
```

GOTCHA `replace("x", "y")` replaces only the first match — a surprisingly common bug. Use `replaceAll("x", "y")` (ES2021) for all of them, or a regex with the global flag `/x/g`. Remember every method returns a new string: `s.trim()` does nothing unless you use the return value (`s = s.trim()`).

WHEN TO USE Cleaning methods exist because text from the real world — forms, files, APIs — rarely arrives in the exact shape you need. `trim` before validating or storing input prevents whitespace bugs; `toLowerCase` before comparing makes matches case-insensitive; `padStart` formats numbers into fixed-width displays (invoice IDs, timers). They all return new strings precisely *because* strings are immutable — which is safe, since the original is never silently altered under another reference.

VERSION `replaceAll` is ES2021; `trimStart` / `trimEnd` are ES2019; `padStart` / `padEnd` are ES2017. All widely supported.

Whitespace & case

.trim() `str.trim()`

Removes whitespace from both ends. Essential for cleaning form input.

.trimStart() / **.trimEnd()** `str.trimStart()`

Trim one side only. **ES2019**.

.toLowerCase() / **.toUpperCase()** `str.toLowerCase()`

Change case — for normalizing before comparison or for display.

.toLocaleLowerCase() `str.toLocaleLowerCase(locale)`

Locale-aware case change (matters for some languages, e.g. Turkish 'i').

.normalize() `str.normalize(form?)`

Normalizes Unicode so visually identical accented text compares equal.

Replace & pad

.replace() `str.replace(target, repl)`

Replaces the **first** match (string) or all (with a `/g` regex). Replacement can be a function.

.replaceAll() `str.replaceAll(target, repl)`

Replaces **every** occurrence. The clearest choice for "swap all." **ES2021**.

.padStart() `str.padStart(len, fill?)`

Pads the start up to `len` (e.g. zero-pad numbers). **ES2017**.

.padEnd() `str.padEnd(len, fill?)`

Pads the end up to `len`.

.repeat() `str.repeat(n)`

Returns the string repeated `n` times.

.concat() `str.concat(...strs)`

Joins strings; in practice template literals or `+` are clearer.

Splitting, joining & padding CORE

IN PLAIN TERMS

A very common need: turn one string into a list of pieces (split a sentence into words, a CSV line into fields), or turn a list back into one string. `split` breaks a string apart on a separator into an array; the array's `join` (from Part 1) puts it back together.

`split(separator)` is the key method — split on a comma for CSV fields, on a space for words, on `""` for individual characters. It returns an array, so it's the bridge from text into the array toolbox you learned in Part 1. Combined with `map` / `filter` and `join`, it's how most text-processing pipelines are built.

▼ KEY TERMS IN THIS SECTION

<code>=></code>	Arrow — defines an <i>arrow function</i> , a short way to write a function (e.g. <code>x => x * 2</code>). The arrow separates the inputs from what the function does.
<code>...</code>	Spread/rest operator (three dots) — either <i>spreads</i> a collection out into its items, or <i>gathers</i> loose items into one collection, depending on where it's used.
<code>regular expression</code>	Regular expression — a pattern for matching text, written between slashes: <code>/abc/</code> .
<code>expression</code>	Expression — any piece of code that produces a value, like <code>2 + 2</code> or <code>user.name</code> .
<code>code point</code>	Code point — the true number of a single character, which may span two code units.
<code>code unit</code>	Code unit — the 16-bit pieces a JavaScript string is made of; most characters are one, some (emoji) are two.
<code>element</code>	Element — a node created from an HTML tag (a <code><div></code> , <code><p></code> , <code><button></code>); the most common kind of node.
<code>spread</code>	Spread — using <code>...</code> to expand a list or object into its individual pieces.

`split` turns text into an array (Part 1's whole toolbox); `join` turns it back. The `split` → `map` → `join` pipeline is the workhorse of text cleaning.

```
"a,b,c".split(",");           // ["a", "b", "c"] - CSV fields
"one two three".split(" ");   // ["one", "two", "three"] - words
"hello".split("");           // ["h","e","l","l","o"] - characters
"a,b,c".split(",", 2);       // ["a", "b"] - limit the count

// a typical pipeline: split → clean → rejoin
const cleaned = " apple, banana ,cherry "
  .split(",")
  .map(s => s.trim())          // remove stray spaces around each
  .filter(Boolean)            // drop any empty pieces
  .join(", ");                // "apple, banana, cherry"

[...("café")];               // ["c","a","f","é"] - spread splits by code point
```

GOTCHA `split("")` breaks on UTF-16 code units, so it can split an emoji in half. To split into true characters, spread the string instead: `[...str]`. And `"".split(",")` returns `[""]` (a one-element array with an empty string), not an empty array — guard for that when processing possibly-empty input.

WHEN TO USE `split` exists as the bridge between the text world and the array world: once text is an array of pieces, the entire Array toolbox (`map`, `filter`, `reduce`, `find`) becomes available. This is why so much data wrangling looks like `text.split(...).map(...).filter(...).join(...)` — parsing CSV lines, tokenizing input, cleaning lists. For splitting on a *pattern* rather than a fixed string (e.g. "any whitespace"), `split` also accepts a regular expression: `str.split(/s+/)`.

Split into pieces

`.split()` `str.split(sep, limit?)`

Breaks a string into an array on each `sep` (string or regex). The gateway into the Array toolbox. Empty `sep` splits per code unit.

`[...str]` spread

Splits into true characters (by code point) — safer than `split("")` for emoji/accents.

`.join()` `arr.join(sep)`

(Array method) Rejoins pieces into a string — the inverse of `split`. See Part 1.

Part 3 • Number & Math

Working with numbers: turning text into numbers safely, formatting them for display, checking for the odd cases (`NaN`, infinity), and the `Math` object for rounding, ranges, and randomness.

Numbers CORE

IN PLAIN TERMS

JavaScript has one number type for everyday use — it covers whole numbers and decimals alike. The catch every developer must know: numbers are stored as **floating point**, so some decimal math is slightly inexact (`0.1 + 0.2` is not exactly `0.3`). This section covers converting text to numbers, formatting numbers back to text, and the safety checks that keep that floating-point reality from biting you.

Convert text to numbers with `Number()` (strict) or `parseInt`/`parseFloat` (lenient). Format numbers for display with `toFixed` (fixed decimals) — though for currency and locale-aware formatting, `Intl.NumberFormat` (later) is better. Guard against bad values with `Number.isNaN` and `Number.isFinite`.

▼ KEY TERMS IN THIS SECTION

- ===** Strict equality operator — compares two values and returns `true` only if they're the same type and value, with no automatic conversion. Your default for comparing.
- ==** Loose equality operator — compares two values *after* converting them to a common type, which causes surprises. Avoid in favour of `===`.
- **** Exponentiation operator — raises a number to a power (`2 ** 3` is `8`).
- coercion** Coercion — JavaScript *automatically* converting a value from one type to another (e.g. number to text).
- BigInt** BigInt — a number type for integers too large for the normal number type to hold exactly.
- locale** Locale — a language/region setting (like `en-US`) that controls formatting of dates, numbers, and text.
- radix** Radix — the number base used when reading text as a number; `10` means ordinary decimal.
- NaN** NaN ("Not a Number") — the result of an invalid math operation, like `0/0`.

`toFixed` returns a *string*, not a number — good for display, but don't keep doing math on the result. For real money handling, store integer cents and format with `Intl`.

```
Number("42");           // 42           strict — whole string must be numeric
Number("42px");          // NaN          (parseInt would give 42)
parseInt("42px", 10);     // 42           leading integer; ALWAYS pass radix 10
parseFloat("3.14em");     // 3.14

(1234.5).toFixed(2);      // "1234.50"   — fixed decimals (returns a STRING)
(0.1 + 0.2).toFixed(2);   // "0.30"      — tames floating-point display

Number.isInteger(42);     // true
Number.isNaN(0/0);        // true        — the correct NaN test
Number.isFinite(1/0);     // false        (1/0 is Infinity)
Number.MAX_SAFE_INTEGER;  // 9007199254740991 — beyond this, use BigInt

(255).toString(16);       // "ff"        — number to hex/binary/other base
```

GOTCHA Use `Number.isNaN`, not the global `isNaN`: the global one coerces first, so `isNaN("hello")` is `true` (misleading), while `Number.isNaN("hello")` is `false` (correct — it's a string, not NaN). Same for `Number.isFinite` over the global `isFinite`. And remember floating point: never compare computed decimals with `===`; compare within a small tolerance, or work in integers.

WHEN TO USE These exist because numeric data constantly crosses the text boundary — form fields, URLs, CSV cells arrive as strings and must become numbers before math. `Number()` is strict (good for validated input where junk should fail); `parseInt` / `parseFloat` are lenient (good for pulling a number out of `"42px"`). The `Number.isX` checks exist to safely handle the special values (`NaN`, `Infinity`) that arithmetic can produce without throwing — essential when you can't trust the input.

Convert & format

Number() `Number(x)`

Strict text→number; whole string must be numeric or you get `NaN`.

parseInt() `parseInt(s, 10)`

Leading integer from text, ignoring trailing junk. Always pass radix `10`.

parseFloat() `parseFloat(s)`

Like `parseInt` but keeps decimals.

.toFixed() `n.toFixed(digits)`

Fixed number of decimals, as a *string*. For display.

.toPrecision() `n.toPrecision(sig)`

Format to a number of significant digits (string).

.toString() `n.toString(radix?)`

Number to text; optional base (`toString(2)` binary, `toString(16)` hex).

.toLocaleString() `n.toLocaleString(locale)`

Locale-formatted number (thousands separators, currency). See `Intl` for full control.

Checks & constants

Number.isNaN() `Number.isNaN(x)`

Correct `NaN` test (no coercion). Prefer over global `isNaN`.

Number.isFinite() `Number.isFinite(x)`

True for real finite numbers; false for `Infinity` / `NaN`. Prefer over global `isFinite`.

Number.isInteger() `Number.isInteger(x)`

True if the value is a whole number.

Number.isSafeInteger() `Number.isSafeInteger(x)`

True if the integer is within the exactly-representable range.

Number.MAX_SAFE_INTEGER constant

Largest exactly-representable integer (`253 - 1`); beyond it, use `BigInt`.

Number.EPSILON constant

Smallest difference between representable numbers — used for tolerant float comparisons.

Math CORE

IN PLAIN TERMS

`Math` is a built-in object full of math helpers — you don't create it, you just call its functions: `Math.round()`, `Math.max()`, `Math.random()`, and so on. It handles rounding, finding the largest/smallest, powers and roots, and generating random numbers.

The everyday members: `round` / `floor` / `ceil` / `trunc` for turning decimals into integers, `min` / `max` for ranges, `abs` for magnitude, `random` for randomness, and `pow` / `sqrt` for powers and roots. All are static — called directly on `Math`.

▼ KEY TERMS IN THIS SECTION

- ...** Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.
- **** Exponentiation operator — raises a number to a power (`2 ** 3` is `8`).
- operator** Operator — a symbol that performs an action on values, like `+` (add) or `===` (compare).
- spread** Spread — using `...` to expand a list or object into its individual pieces.
- static** Static member — a property or method that belongs to the class itself, not to individual instances.

`Math.random()` gives a float in $[0, 1)$. The `Math.floor(Math.random() * n)` pattern turns it into a random integer — memorize it.

```
Math.round(4.5);    // 5    nearest integer
Math.floor(4.9);    // 4    round DOWN
Math.ceil(4.1);     // 5    round UP
Math.trunc(-4.7);   // -4   drop the decimal (no rounding)
```

```
Math.max(3, 7, 2);  // 7
Math.min(...[3, 7, 2]); // 2    spread an array in
Math.abs(-5);       // 5
```

```
Math.pow(2, 10);    // 1024 (same as 2 ** 10)
Math.sqrt(144);     // 12
```

```
// random integer in a range [min, max] inclusive:
const roll = Math.floor(Math.random() * 6) + 1; // 1..6
Math.random(); // 0 (inclusive) to 1 (exclusive)
```

GOTCHA `Math.round` rounds .5 up (toward $+\infty$), so `Math.round(-2.5)` is `-2`, not `-3` — occasionally surprising for negatives. `Math.max()` with no arguments returns `-Infinity` (and `Math.min()` returns `Infinity`), so guard against empty inputs. And `Math.random()` is **not** cryptographically secure — use `crypto.getRandomValues` for security-sensitive randomness.

WHEN TO USE `Math` is a static utility object (not something you instantiate) because these operations are pure functions on numbers — there's no state to hold. It exists so common numeric operations are built-in and fast rather than hand-rolled. The rounding family is the most-used: `floor` for "how many complete groups," `ceil` for "how many containers needed," `round` for display, `trunc` to simply drop decimals. `min` / `max` with spread (`Math.max(...arr)`) is the quick way to find array extremes without a loop.

Rounding

Math.round() `Math.round(n)`

Nearest integer (.5 rounds up).

Math.floor() `Math.floor(n)`

Rounds down. Used in the random-integer idiom and "how many full groups."

Math.ceil() `Math.ceil(n)`

Rounds up. "How many containers needed."

Math.trunc() `Math.trunc(n)`

Drops the fractional part without rounding. **ES2015**.

Math.sign() `Math.sign(n)`

Returns -1, 0, or 1 for the sign of a number.

Ranges, powers, random

Math.max() / **Math.min()** `Math.max(...nums)`

Largest / smallest of the arguments. Spread an array to use on a list.

Math.abs() `Math.abs(n)`

Absolute value (distance from zero).

`Math.pow()` `Math.pow(base, exp)`

Power; the `**` operator is usually clearer.

`Math.sqrt()` / `Math.cbrt()` `Math.sqrt(n)`

Square / cube root.

`Math.random()` `Math.random()`

Random float in [0, 1). Combine with `floor` for integers. Not for security.

`Math.hypot()` `Math.hypot(a, b)`

Square root of the sum of squares — distance between points.

`Math.trig` / `log` `Math.sin`, `Math.log`, ...

Trigonometry and logarithms — `sin`, `cos`, `tan`, `log`, `log2`, `log10`, `exp`, and constants like `Math.PI`.

Part 4 • Object

Beyond the literal syntax from Book 1, the `Object` built-in provides static helper methods for reading an object's contents, building objects from data, merging them, and locking them down. These are the tools for treating objects as data you inspect and transform.

Reading, building & merging objects CORE

IN PLAIN TERMS

An **object** holds named values (`{name: "Ada", age: 36}`). Often you need to work with all its keys or values at once — list them, loop over them, build an object from a list of pairs, or combine two objects. The `Object.*` static methods (called on `Object` itself, not on your object) do this.

`Object.keys` / `values` / `entries` turn an object into arrays you can use with the Array toolbox — this is how you *iterate* an object. `Object.fromEntries` builds an object back from pairs (the inverse of `entries`). Spread `{...a, ...b}` and `Object.assign` merge objects. Together they let you transform objects as fluidly as arrays.

▼ KEY TERMS IN THIS SECTION

<code>=></code>	Arrow — defines an <i>arrow function</i> , a short way to write a function (e.g. <code>x => x * 2</code>). The arrow separates the inputs from what the function does.
<code>...</code>	Spread/rest operator (three dots) — either <i>spreads</i> a collection out into its items, or <i>gathers</i> loose items into one collection, depending on where it's used.
<code>**</code>	Exponentiation operator — raises a number to a power (<code>2 ** 3</code> is <code>8</code>).
static method	Static method — a method called on the type itself (<code>Array.from</code> , <code>Object.keys</code>), not on an instance.
immutable	Immutable update — making a new value with the change instead of altering the original.
prototype	Prototype — the object a JavaScript object inherits shared properties and methods from.
argument	Argument — the actual value you pass into a function when you call it.
iterable	Iterable — any value you can step through one item at a time with <code>for...of</code> (arrays, strings, Maps, Sets).
shallow	Shallow copy — a copy one level deep; nested objects are still shared with the original.
spread	Spread — using <code>...</code> to expand a list or object into its individual pieces.
static	Static member — a property or method that belongs to the class itself, not to individual instances.
Symbol	Symbol — a guaranteed-unique value, used mainly as a special, collision-proof object key.

The `entries` → `map` → `fromEntries` round-trip is the standard way to transform an object's keys/values — the object equivalent of a `map` over an array.

```
const user = {name: "Ada", role: "admin", active: true};

Object.keys(user);    // ["name", "role", "active"]
Object.values(user);  // ["Ada", "admin", true]
Object.entries(user); // [["name", "Ada"], ["role", "admin"], ["active", true]]

// iterate an object by turning it into entries:
for (const [key, value] of Object.entries(user)) {
  console.log(`${key}: ${value}`);
}

// transform an object: entries → map → fromEntries
const upper = Object.fromEntries(
  Object.entries(user).map(([k, v]) => [k, String(v).toUpperCase()])
);

// merge (later keys win):
const patched = {...user, role: "owner"}; // spread (immutable)
const merged = Object.assign({}, defaults, overrides); // into a new object

Object.groupBy([1,2,3,4], n => n % 2 ? "odd" : "even");
// { odd: [1,3], even: [2,4] } (ES2024)
```

GOTCHA `Object.keys` and friends only see **own, enumerable, string** keys — not inherited ones and not Symbol keys. For merging, both spread and `Object.assign` are **shallow**: nested objects are shared, and `Object.assign` *mutates* its first argument (pass a fresh `{}` as the target to avoid surprises). Use `structuredClone` for deep copies.

WHEN TO USE These exist to bridge objects into the array world: an object isn't directly iterable with `for...of` or `map`, but `Object.entries(obj)` gives you `[key, value]` pairs that *are*, unlocking the whole Array toolbox for transforming records. `fromEntries` closes the loop, letting you rebuild an object after transforming. This is *why* reshaping API data — renaming keys, filtering fields, converting values — is usually written as `fromEntries(entries(obj).map(...))`. `Object.groupBy` (ES2024) exists because grouping rows by a category is so common it deserved a built-in instead of a hand-written `reduce`.

VERSION `Object.entries` / `values` are **ES2017**; `Object.fromEntries` is **ES2019**; `Object.hasOwn` is **ES2022**; `Object.groupBy` is **ES2024** (originally proposed as `Array.prototype.group`, moved to a static method for web compatibility).

Read contents

Object.keys() `Object.keys(obj)`

Array of own enumerable string keys. The basis for iterating an object.

Object.values() `Object.values(obj)`

Array of the corresponding values.

Object.entries() `Object.entries(obj)`

Array of `[key, value]` pairs — use with `for...of`, `map`, etc.

Build & merge

Object.fromEntries() `Object.fromEntries(pairs)`

Builds an object from `[key, value]` pairs. Inverse of `entries`. ES2019.

{...spread} `object spread`

Copy/merge into a new object; later keys win. The immutable-update idiom. ES2018.

Object.assign() `Object.assign(target, ...src)`

Copies own enumerable props into `target` (mutates it). Pass `{}` as target for a merge into a new object.

Object.groupBy() `Object.groupBy(items, keyFn)`

Groups items into an object keyed by `keyFn`'s return. Replaces a manual `reduce`. ES2024.

Protecting & inspecting objects REFERENCE

IN PLAIN TERMS

Sometimes you want to lock an object so it can't be changed (a constant config), or ask low-level questions about it (does it *really* own this key, or is it inherited?). These methods handle protection and inspection.

`Object.freeze` makes an object read-only (shallowly). `Object.hasOwn` checks for an own (non-inherited) property — the modern replacement for `hasOwnProperty`. The rest (`create`, `getPrototypeOf`, `defineProperty`) are lower-level prototype tools you'll rarely need directly now that classes exist.

▼ KEY TERMS IN THIS SECTION

- **** Exponentiation operator — raises a number to a power (`2 ** 3` is `8`).
- strict mode** Strict mode — a stricter dialect that turns silent mistakes into clear errors; automatic in modules and classes.
- prototype** Prototype — the object a JavaScript object inherits shared properties and methods from.
- property** Property — a value on the live element object in JavaScript; may stay in sync with an attribute or diverge from it.
- shallow** Shallow copy — a copy one level deep; nested objects are still shared with the original.
- static** Static member — a property or method that belongs to the class itself, not to individual instances.
- rest** Rest — using `...` to collect several values into a single array or object.

`Object.hasOwn(obj, key)` answers "does this object itself have this key?" — unlike `in`, which also returns true for inherited keys like `toString`.

```
const config = Object.freeze({apiUrl: "/api", retries: 3});
config.retries = 5;           // silently ignored (throws in strict mode)
Object.isFrozen(config);     // true

Object.hasOwn(user, "role"); // true - own property? (replaces hasOwnProperty)
"toString" in user;         // true - but `in` also sees INHERITED keys
Object.hasOwn(user, "toString"); // false - own only

Object.getPrototypeOf(user); // the object it inherits from
Object.create(proto);        // new object with a given prototype
```

WHEN TO USE `Object.freeze` exists to turn "this shouldn't change" from a comment into an enforced rule — handy for configuration objects and constants, catching accidental writes (loudly, in strict mode) instead of letting them corrupt state. `Object.hasOwn` exists because the old `obj.hasOwnProperty(key)` could break if an object had a property literally named `hasOwnProperty`, or was created without the usual prototype; the static `Object.hasOwn(obj, key)` is always safe. The

prototype-level tools (`create`, `getPrototypeOf`, `defineProperty`) exist for library authors and advanced inheritance work — application code rarely touches them directly.

Protect

`Object.freeze()` `Object.freeze(obj)`

Makes an object read-only (shallow). Writes are ignored / throw in strict mode.

`Object.isFrozen()` `Object.isFrozen(obj)`

Tests whether an object is frozen.

`Object.seal()` `Object.seal(obj)`

Prevents adding/removing keys but allows editing existing ones.

Inspect & low-level

`Object.hasOwn()` `Object.hasOwn(obj, key)`

Safe own-property check. Replaces `hasOwnProperty`. ES2022.

`Object.getPrototypeOf()` `Object.getPrototypeOf(obj)`

Returns the object's prototype.

`Object.create()` `Object.create(proto, props?)`

Creates a new object with a specified prototype. Pre-class inheritance tool.

`Object.defineProperty()` `Object.defineProperty(obj, key, desc)`

Fine-grained property control (writable/enumerable/getters). For libraries.

`Object.getOwnPropertyNames()` `(obj)`

All own string keys, including non-enumerable ones.

Part 5 • Keyed Collections

When a plain object isn't the right shape for your data, these purpose-built collections are:

`Map` for key–value pairs with any kind of key, `Set` for unique values, and their `Weak` variants for memory-friendly associations.

Map CORE

IN PLAIN TERMS

A **Map** is a collection of key→value pairs, like an object — but better suited when you're using it as a *dictionary* you constantly add to and remove from. Unlike objects, a Map's keys can be **any type** (objects, numbers, not just strings), it remembers **insertion order**, it has a `.size`, and it's built for frequent changes. You create one with `new Map()`.

Use `.set(key, value)` to add, `.get(key)` to read, `.has(key)` to check, `.delete(key)` to remove, and `.size` for the count. A Map is iterable, so `for...of` gives you `[key, value]` pairs directly. It's the right tool when keys aren't simple strings or when you're building a lookup table on the fly.

▼ KEY TERMS IN THIS SECTION

<code>===</code>	Strict equality operator — compares two values and returns <code>true</code> only if they're the same type and value, with no automatic conversion. Your default for comparing.
<code>==</code>	Loose equality operator — compares two values <i>after</i> converting them to a common type, which causes surprises. Avoid in favour of <code>===</code> .
<code>=></code>	Arrow — defines an <i>arrow function</i> , a short way to write a function (e.g. <code>x => x * 2</code>). The arrow separates the inputs from what the function does.
<code>...</code>	Spread/rest operator (three dots) — either <i>spreads</i> a collection out into its items, or <i>gathers</i> loose items into one collection, depending on where it's used.
<code>**</code>	Exponentiation operator — raises a number to a power (<code>2 ** 3</code> is <code>8</code>).
constructor	Constructor — the special method that sets up a new instance when you use <code>new</code> .
prototype	Prototype — the object a JavaScript object inherits shared properties and methods from.
operator	Operator — a symbol that performs an action on values, like <code>+</code> (add) or <code>===</code> (compare).
iterable	Iterable — any value you can step through one item at a time with <code>for...of</code> (arrays, strings, Maps, Sets).
property	Property — a value on the live element object in JavaScript; may stay in sync with an attribute or diverge from it.
index	Index — an item's numbered position in an array or string, starting at 0.
NaN	NaN ("Not a Number") — the result of an invalid math operation, like <code>0/0</code> .

`set` returns the Map, so calls chain: `m.set("a",1).set("b",2)`. Convert to/from objects with `Object.fromEntries(map)` and `new Map(Object.entries(obj))`.

```
const cache = new Map();

cache.set("user:1", {name: "Ada"}); // add / update
cache.get("user:1");                // {name: "Ada"}
cache.has("user:1");                // true
cache.size;                         // 1
cache.delete("user:1");              // true (removed)

// keys can be ANY type – even objects:
const byElement = new Map();
byElement.set(domNode, {clicks: 0}); // object as a key – impossible with {}

// build from pairs, and iterate in insertion order:
const m = new Map([["a", 1], ["b", 2]]);
for (const [key, value] of m) { /* ... */ }
m.forEach((value, key) => { /* ... */ });

[...m.keys()]; // ["a", "b"]
[...m.values()]; // [1, 2]
Object.fromEntries(m); // {a:1, b:2} – convert Map → object
```

GOTCHA Map keys use the same identity rules as `===`, so two different objects are different keys even if they look identical — and `NaN` is usable as a key (treated as equal to itself). Don't reach for the index operator: it's `map.get(k)`, not `map[k]` (the latter sets a normal property and bypasses the Map).

WHEN TO USE `Map` exists for the jobs plain objects do badly. Objects force keys to strings (or symbols), inherit keys from their prototype (so `"constructor" in obj` is unexpectedly true), have no built-in size, and aren't optimized for constant add/remove. A Map fixes all of that: any key type, no inherited surprises, reliable order, `.size`, and fast updates. **Rule of thumb:** if your keys are dynamic, non-string, or the set of keys changes a lot at runtime (a cache, a lookup table, counting occurrences), use a Map; if you have a fixed, known set of named fields (a record/struct), a plain object is simpler.

Map operations

new Map() new Map(entries?)

Creates a Map, optionally from `[key, value]` pairs. Keys may be any type.

.set() map.set(key, value)

Adds or updates a pair. Returns the Map (chainable).

.get() map.get(key)

Returns the value for a key, or `undefined`. Use this, not `map[key]`.

.has() map.has(key)

`true` if the key exists.

.delete() map.delete(key)

Removes a pair; returns whether it existed.

.size map.size

Number of pairs (a property, not a method).

`.clear()` `map.clear()`

Removes everything.

`.forEach()` `map.forEach((v, k) => ...)`

Runs a function per pair (value first, then key).

`.keys()` / `.values()` / `.entries()` `iterators`

Iterators over keys / values / `[k, v]` pairs. The Map itself iterates as entries.

Set CORE

IN PLAIN TERMS

A **Set** is a collection of **unique** values — it automatically rejects duplicates. If you add the same value twice, it's still only there once. This makes it perfect for de-duplicating a list, or tracking "which things have I seen / selected?" You create one with `new Set()`.

Use `.add(value)`, `.has(value)`, `.delete(value)`, and `.size`. The single most common use is removing duplicates from an array: `[...new Set(arr)]`. ES2025 added set-algebra methods — `union`, `intersection`, `difference` — for combining sets directly.

▼ KEY TERMS IN THIS SECTION

`===` Strict equality operator — compares two values and returns `true` only if they're the same type and value, with no automatic conversion. Your default for comparing.

`==` Loose equality operator — compares two values *after* converting them to a common type, which causes surprises. Avoid in favour of `===`.

`...` Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.

`**` Exponentiation operator — raises a number to a power (`2 ** 3` is `8`).

`iterable` Iterable — any value you can step through one item at a time with `for...of` (arrays, strings, Maps, Sets).

`[...new Set(array)]` is the idiomatic one-liner for removing duplicates. Membership tests (`has`) are much faster than `array.includes` for large collections.

```
const seen = new Set();
seen.add("a");
seen.add("a");           // ignored – already present
seen.has("a");           // true
seen.size;               // 1

// THE classic use – dedupe an array:
const unique = [...new Set([1, 2, 2, 3, 3, 3])]; // [1, 2, 3]

// track selections:
const selected = new Set();
selected.add(id); selected.delete(id);
selected.has(id);        // membership test – very fast

// set algebra (ES2025):
const a = new Set([1, 2, 3]), b = new Set([2, 3, 4]);
a.intersection(b);       // Set {2, 3}
a.union(b);              // Set {1, 2, 3, 4}
a.difference(b);         // Set {1}
```

GOTCHA Set uniqueness uses `===`, so two objects with identical contents are still two distinct members (`new Set([x:1], {x:1}]).size` is `2`). It dedupes *primitives* perfectly but not equivalent objects. The set-algebra methods (`union`, etc.) are ES2025 — check runtime support before relying on them.

WHEN TO USE `Set` exists for two jobs that arrays handle awkwardly: guaranteeing uniqueness and testing membership quickly. De-duping with an array means `filter` + `indexOf` (slow, $O(n^2)$); a `Set` does it in one pass. Checking "have I seen this?" with `array.includes` scans the whole array each time; `set.has` is near-instant regardless of size. So whenever you're tracking a collection of distinct things — visited IDs, selected rows, unique tags — a `Set` is both clearer and faster. The new set-algebra methods exist so combining sets (shared tags, missing items) reads as one call instead of a manual loop.

VERSION `Set` and `Map` are ES2015. The set methods `union`, `intersection`, `difference`, `symmetricDifference`, `isSubsetOf`, `isSupersetOf`, `isDisjointFrom` are ES2025.

Set operations

new Set() `new Set(iterable?)`

Creates a `Set`, optionally from an iterable (dropping duplicates).

.add() `set.add(value)`

Adds a value if not present. Returns the `Set` (chainable).

.has() `set.has(value)`

Fast membership test. Much faster than `array.includes` at scale.

.delete() `set.delete(value)`

Removes a value; returns whether it existed.

.size `set.size`

Number of values.

.clear() `set.clear()`

Removes everything.

`.values()` / `.keys()` / `.entries()` iterators

Iterators (keys === values for a Set). The Set itself iterates its values.

`.union()` / `.intersection()` / `.difference()` set algebra

Combine sets: all / shared / only-in-this. Plus `symmetricDifference`, `isSubsetOf`, `isSupersetOf`, `isDisjointFrom`. ES2025.

WeakMap & WeakSet REFERENCE

IN PLAIN TERMS

These are special, memory-friendly versions of Map and Set. The key idea: they hold their keys **weakly**, meaning if nothing else in your program references that key object, it can be cleaned up (garbage-collected) automatically — and its entry disappears. This prevents memory leaks when you're associating extra data with objects you don't fully control.

A `WeakMap` lets you attach private data to an object without preventing that object from being freed when it's no longer used elsewhere — useful for caches and metadata keyed by DOM nodes or class instances. They're not iterable and have no `.size` (by design), so they're a specialized tool, not a general Map replacement.

▼ KEY TERMS IN THIS SECTION

**	Exponentiation operator — raises a number to a power (<code>2 ** 3</code> is <code>8</code>).
instance	Instance — a specific object built from a class blueprint.
iterable	Iterable — any value you can step through one item at a time with <code>for...of</code> (arrays, strings, Maps, Sets).
element	Element — a node created from an HTML tag (a <code><div></code> , <code><p></code> , <code><button></code>); the most common kind of node.
rest	Rest — using <code>...</code> to collect several values into a single array or object.
node	Node — any single point in the DOM tree: an element, a piece of text, a comment, or the document itself.
DOM	DOM (Document Object Model) — the browser's live, in-memory tree of objects representing the page; changing it changes what's on screen.

Keys must be objects (not primitives). The trade-off for automatic cleanup is that you can't list or count the contents.

```
const metadata = new WeakMap();

function track(element) {
  metadata.set(element, {created: Date.now()}); // attach data to a DOM node
}
// when `element` is removed and forgotten elsewhere,
// its WeakMap entry is cleaned up automatically — no leak.

metadata.get(element); // the data, if element still exists
metadata.has(element); // true/false
// no .size, no iteration — keys can vanish at any time
```

WHEN TO USE `WeakMap` / `WeakSet` exist to solve a specific memory problem: if you store data about objects in a normal `Map`, that Map keeps those objects alive forever (the Map references them), causing leaks in long-running apps. A `WeakMap` references its keys *weakly*, so once the rest of your program is done with an object, both it and its `WeakMap` entry are reclaimed automatically. Use **them** for caches, private per-instance data, or metadata keyed on DOM nodes — anywhere you'd otherwise risk holding objects in memory longer than needed. For ordinary key-value work where you want to iterate or count, use a regular Map.

Weak collections

`new WeakMap()` `new WeakMap()`

Map with weakly-held **object** keys, auto-cleaned when keys are otherwise unreferenced. `set` / `get` / `has` / `delete` only.

`new WeakSet()` `new WeakSet()`

Set of weakly-held objects. `add` / `has` / `delete` only.

(no `size` / `iteration`) by design

Weak collections can't be enumerated or counted, because entries may disappear at any time.

Part 6 • JSON

JSON is the universal text format for data exchange — what APIs send, what you save to storage, what config files use. Two methods do all the work: one turns data into JSON text, the other turns JSON text back into data.

JSON CORE

IN PLAIN TERMS

JSON (JavaScript Object Notation) is a way of writing data as plain text so it can be sent over the internet or saved to a file. It looks almost exactly like a JavaScript object. The catch: a live JavaScript object lives in memory, while JSON is *text*. You **stringify** to go from object → text (to send or store it), and **parse** to go from text → object (when it comes back).

`JSON.stringify(value)` converts data to a JSON string. `JSON.parse(text)` converts a JSON string back to data. This is the boundary you cross every time you talk to a server (`fetch`), use `localStorage`, or read a config file. Both have optional extra arguments for formatting and transforming.

▼ KEY TERMS IN THIS SECTION

- **** Exponentiation operator — raises a number to a power (`2 ** 3` is `8`).
- shallow** Shallow copy — a copy one level deep; nested objects are still shared with the original.
- module** Module — a single code file with its own scope that shares code via `import` / `export`.
- import** Import — bringing in something that another file has exported.
- parse** Parse — read text and turn it into structured data (e.g. `JSON.parse`).
- NaN** NaN ("Not a Number") — the result of an invalid math operation, like `0/0`.

The third argument to `stringify` is indentation — `JSON.stringify(x, null, 2)` produces readable, multi-line JSON for files and debugging.

```
const user = {name: "Ada", roles: ["admin"], active: true};

const text = JSON.stringify(user);
// '{"name":"Ada","roles":["admin"],"active":true}' - ready to send/store

const back = JSON.parse(text);
// {name: "Ada", roles: ["admin"], active: true} - a real object again

// pretty-print with 2-space indentation (great for files/logs):
JSON.stringify(user, null, 2);

// the everyday round-trips:
localStorage.setItem("user", JSON.stringify(user));
const saved = JSON.parse(localStorage.getItem("user"));

// a quick (shallow-ish) deep clone of plain data:
const copy = JSON.parse(JSON.stringify(user));
```

GOTCHA `JSON.parse` throws on invalid JSON — always wrap it in `try/catch` when the text comes from outside (a network response, user input, storage). Also, JSON can't represent everything: `undefined`, functions, and Symbols are dropped by `stringify`; `Date` objects become strings (and don't auto-revive on parse); and `NaN` / `Infinity` become `null`. The `JSON.parse(JSON.stringify(x))` clone trick works only for plain JSON-safe data — use `structuredClone` for anything richer.

WHEN TO USE JSON exists because programs need to exchange data as text — over a network, into a file, through `localStorage` — but your code works with live objects in memory. `stringify` / `parse` are the translators between those two worlds, and JSON won as *the* format because it's human-readable, language-independent, and maps almost directly onto JavaScript's own objects and arrays. This is *why* nearly every API speaks JSON and why these two methods are among the most-called in any real app: every save, every load, every API call crosses this boundary. The optional `replacer` / `reviver` functions exist for the cases where you need to customize the translation — omitting secret fields when sending, or reviving date strings back into `Date` objects when receiving.

The two methods

JSON.stringify() `JSON.stringify(value, replacer?, space?)`

Data → JSON text. `replacer` filters/transforms; `space` adds indentation for readability.

JSON.parse() `JSON.parse(text, reviver?)`

JSON text → data. **Wrap in try/catch** for untrusted input. `reviver` transforms values as they're parsed (e.g. revive dates).

toJSON() `obj.toJSON()`

If an object defines this method, `stringify` uses its return value — how `Date` serializes to an ISO string.

JSON modules import data from `"./d.json"` with `{type:"json"}`

Import a JSON file directly as a module. **ES2025** (import attributes).

Part 7 • Dates & Internationalization

Working with time and with formatting for human readers. `Date` represents moments in time; `Intl` formats dates, numbers, and currency correctly for any language and region.

Date CORE

IN PLAIN TERMS

A `Date` is JavaScript's object for a single moment in time. Internally it's just a number: the count of milliseconds since a fixed reference point called the **epoch** (midnight, Jan 1 1970, UTC). You create one with `new Date()`. Working with dates is notoriously fiddly, so this section focuses on the safe, common operations and flags the traps.

`new Date()` is now; `new Date(ms)` or `new Date("2024-01-05")` is a specific moment. You read parts with `getFullYear`, `getMonth`, etc., and get a sortable/storable string with `toISOString`. For *displaying* dates to users, don't hand-format — use `Intl.DateTimeFormat` (next topic), which handles locales correctly.

▼ KEY TERMS IN THIS SECTION

**	Exponentiation operator — raises a number to a power (<code>2 ** 3</code> is <code>8</code>).
reference	Reference — a pointer to an object; copying it copies the pointer, so two names can share one object.
mutate	Mutate — to change a value in place; a mutating method alters the original array/object.
locale	Locale — a language/region setting (like <code>en-US</code>) that controls formatting of dates, numbers, and text.
epoch	Epoch — the reference moment (Jan 1 1970 UTC) that <code>Date</code> timestamps count milliseconds from.
parse	Parse — read text and turn it into structured data (e.g. <code>JSON.parse</code>).

Store and transmit dates as *ISO strings* (`toISOString`) — they're unambiguous and sortable. Subtracting two dates gives milliseconds, which you divide to get other units.

```
const now = new Date();           // current date & time
const d = new Date("2024-01-05"); // from an ISO string (parsed as UTC)
const d2 = new Date(2024, 0, 5);  // year, MONTH (0-based!), day → Jan 5
const fromMs = new Date(1704412800000);

now.getFullYear(); // 2024
now.getMonth();    // 0-11 (0 = January — a classic gotcha)
now.getDate();     // day of month 1-31
now.getDay();      // day of WEEK 0-6 (0 = Sunday)
now.getHours();    // 0-23

Date.now();        // current time as a number (ms) — great for timing
now.toISOString(); // "2024-01-05T00:00:00.000Z" — sortable, storable, UTC
now.getTime();     // milliseconds since epoch

// difference between two dates, in days:
const days = (d2 - d) / (1000 * 60 * 60 * 24);
```

GOTCHA Months are **0-based** (`getMonth()` returns 0 for January), but days of the month are 1-based — endlessly confusing. Also, `new Date("2024-01-05")` is parsed as **UTC**, while `new Date("2024/01/05")` or `new Date(2024, 0, 5)` is **local time** — so the same-looking date can shift by a day depending on timezone. For anything beyond basic needs, a date library (date-fns, Luxon) or the newer `Temporal` API avoids these traps.

WHEN TO USE `Date` exists to represent and compute with moments in time — timestamps on records, scheduling, measuring durations. Its quirks (0-based months, timezone-dependent parsing, mutable setters) come from a rushed 1995 design copied from Java, which is *why* the ecosystem leans on libraries and why TC39 is standardizing a modern replacement (`Temporal`). For now: use `Date.now()` and `getTime()` for timing and math (plain numbers, no quirks), `toISOString()` for storage/transmission, and `Intl` for display. Avoid hand-building human-readable date strings — that's what `Intl` is for.

Create & read

new Date() `new Date(arg?)`

Now (no arg), or a moment from ms / ISO string / `(y, m, d, ...)` parts (month 0-based).

Date.now() `Date.now()`

Current time as milliseconds since epoch. Best for timing and timestamps.

.getTime() `date.getTime()`

This date as ms since epoch. Subtract two to get a duration in ms.

.getFullYear() / getMonth() / getDate() getters

Year, month (0-11), day-of-month (1-31). Local time.

.getDay() `date.getDay()`

Day of week (0 = Sunday).

.getHours() / getMinutes() / getSeconds() getters

Time components, local.

UTC getters `.getUTCFullYear(), ...`

Same readouts in UTC instead of local time.

Format & convert

.toISOString() `date.toISOString()`

Sortable, storable UTC string (`2024-01-05T00:00:00.000Z`). Use for storage/transmission.

Date.parse() `Date.parse(str)`

Parses a date string to ms. Inconsistent across formats — prefer explicit construction.

.toLocaleDateString() `date.toLocaleDateString(locale, opts)`

Locale-aware display string. `Intl.DateTimeFormat` gives more control (next).

setters `.setFullYear(), ...`

Mutate the date in place. Easy to misuse; prefer building a new Date or a library.

Intl CORE

IN PLAIN TERMS

Intl (Internationalization) is the built-in toolbox for formatting things the way humans in different places expect — dates, numbers, currency, lists. A price is `$1,234.50` in the US but `1.234,50 €` in Germany; **Intl** handles all those differences for you instead of you hand-coding them. You use it by creating a formatter and calling `.format()`.

The two you'll use most: **Intl.NumberFormat** (numbers, currency, percentages) and **Intl.DateTimeFormat** (dates and times). You give it a **locale** (like `"en-US"`) and options, and it returns a formatter whose `.format()` does the work — correctly, for that region.

▼ KEY TERMS IN THIS SECTION

- **** Exponentiation operator — raises a number to a power (`2 ** 3` is `8`).
- Symbol** Symbol — a guaranteed-unique value, used mainly as a special, collision-proof object key.
- locale** Locale — a language/region setting (like `en-US`) that controls formatting of dates, numbers, and text.

Reuse a formatter if you're formatting many values — creating one is the expensive part. **Intl** is built into the browser and Node, so there's nothing to install.

```
// currency – correct symbol, separators, and decimal places per locale
new Intl.NumberFormat("en-US", {style: "currency", currency: "USD"})
  .format(1234.5);           // "$1,234.50"
new Intl.NumberFormat("de-DE", {style: "currency", currency: "EUR"})
  .format(1234.5);           // "1.234,50 €"

// plain numbers with grouping, percentages:
new Intl.NumberFormat("en-US").format(1234567);           // "1,234,567"
new Intl.NumberFormat("en-US", {style: "percent"}).format(0.42); // "42%"

// dates, human-readable, locale-correct:
new Intl.DateTimeFormat("en-US", {dateStyle: "long"})
  .format(new Date());           // "January 5, 2024"

// relative time ("3 days ago") and lists ("A, B, and C"):
new Intl.RelativeTimeFormat("en").format(-3, "day");       // "3 days ago"
new Intl.ListFormat("en").format(["A", "B", "C"]);           // "A, B, and C"
```

WHEN TO USE **Intl** exists so you never hand-roll locale formatting — which is deceptively hard (thousands separators, currency symbol placement, decimal commas vs points, date order, pluralization all vary by region) and easy to get wrong. Before **Intl**, apps shipped buggy custom formatters or heavy libraries; now correct, localized output is built into the platform. Use it for any value a user reads: prices (**currency**), counts (grouping), dates, percentages, "time ago" labels, and grammatical lists. Even for a single-locale app, **Intl.NumberFormat** is the cleanest way to get thousands separators and fixed currency formatting — far better than manual **toFixed** plus string surgery.

Formatters

Intl.NumberFormat `new Intl.NumberFormat(locale, opts)`

Numbers, currency, percent, units. `.format(n)` → localized string. The most-used formatter.

Intl.DateTimeFormat `new Intl.DateTimeFormat(locale, opts)`

Dates and times, locale-correct. Use `dateStyle` / `timeStyle` or fine-grained options.

Intl.RelativeTimeFormat `new Intl.RelativeTimeFormat(locale)`

"3 days ago", "in 2 hours" — localized relative time.

Intl.ListFormat `new Intl.ListFormat(locale)`

Grammatical lists: "A, B, and C".

Intl.Collator `new Intl.Collator(locale)`

Locale-aware string sorting/comparison (handles accents correctly).

Intl.PluralRules `new Intl.PluralRules(locale)`

Decide singular/plural forms for a count, per language.

Part 8 • RegExp

Regular expressions are a compact language for describing text patterns — matching, validating, and extracting. They're powerful but terse; this part gives you the working core and points to where the deep detail lives.

Regular expressions REFERENCE

IN PLAIN TERMS

A **regular expression** (regex) is a pattern for matching text — a way to say "a string that looks like *this*" without writing it all out. You write one between slashes: `/\d+/` means "one or more digits." Regexes can check if text matches a shape (is this a valid email?), find parts of it, or replace patterns. They're famously cryptic, so the goal here is the practical core, not mastery of every symbol.

You'll use regexes mainly in three string methods: `test` (does it match? true/false), `match` / `matchAll` (extract what matched), and `replace` / `replaceAll` (swap matches out). The pattern itself is built from literal characters plus special symbols for "any digit," "one or more," "start of string," and so on. Flags after the closing slash (`g` global, `i` case-insensitive) modify behavior.

▼ KEY TERMS IN THIS SECTION

- **** Exponentiation operator — raises a number to a power (`2 ** 3` is `8`).
- ?:** Conditional (ternary) operator — a one-line if/else that produces a value: `condition ? valueIfTrue : valueIfFalse`.
- reference** Reference — a pointer to an object; copying it copies the pointer, so two names can share one object.
- iterator** Iterator — an object that produces values one at a time via `.next()`; what `keys` / `values` / `entries` return.
- Unicode** Unicode — the standard that assigns a number to every character; JavaScript strings are sequences of UTF-16 code units.
- syntax** Syntax — the grammar rules for how code must be written so the computer can understand it.
- Symbol** Symbol — a guaranteed-unique value, used mainly as a special, collision-proof object key.

Build patterns from small pieces: `\d` (digit), `+` (one or more), `^`/`$` (start/end), `()` (capture a piece). The `g` flag means "find all," `i` means "ignore case."

```
const re = /\d{3}-\d{4}/;           // pattern: 3 digits, dash, 4 digits
re.test("555-1234");                // true  - does it match?
re.test("phone: 555-1234 ok");       // true  - match found anywhere

"a1b2c3".match(/\d/g);              // ["1","2","3"] - all digits (g = global)
"2024-01-05".replace(/-/g, "/");    // "2024/01/05" - replace all dashes

// capture groups () pull out pieces:
const m = "2024-01-05".match(/(\d{4})-(\d{2})-(\d{2})/);
m[1]; // "2024"  m[2]; // "01"  m[3]; // "05"

// named groups are clearer:
const nm = "2024-01-05".match(/(?<year>\d{4})-(?<month>\d{2})/);
nm.groups.year; // "2024"

// common building blocks:
// \d digit \w word char \s whitespace . any char
// + one-or-more * zero-or-more ? optional {n} exactly n
// ^ start $ end [abc] any of (a|b) a or b
```

GOTCHA Two frequent traps: (1) a regex with the `g` flag is **stateful** when used with `.test()` / `.exec()` in a loop — it remembers position via `lastIndex`, causing every-other-call bugs; create a fresh regex or avoid `g` with `test`. (2) Don't try to validate complex formats (emails, URLs, HTML) with a regex — they're famously unreliable for that; use a dedicated validator or the browser's built-in input validation. Use regex for *simple, well-defined* patterns.

WHEN TO USE Regular expressions exist to describe text patterns far more compactly than hand-written character-by-character checks — "three digits, a dash, four digits" is `/\d{3}-\d{4}/` instead of a dozen lines of `charAt` logic. They shine for **finding**, **extracting** (with capture groups), and **replacing** structured substrings: pulling fields out of a line, swapping all occurrences of a pattern, splitting on flexible separators. **When to use:** reach for a regex when the thing you're matching is a *variable pattern* (digits, optional parts, repetition); stick with plain string methods (`includes`, `startsWith`, `split`) when you're matching *fixed text*, since they're clearer and faster. For exhaustive syntax, this is a topic worth a dedicated reference — here you have the working subset that covers most real needs.

VERSION Named capture groups `(?<name>...)` are **ES2018**; the `s` (dotAll) flag is **ES2018**; lookbehind is **ES2018**; the `d` (indices) flag is **ES2022**; the `v` (unicodeSets) flag is **ES2024**.

How you use them (string/regex methods)

.test() `regex.test(str)`

`true` / `false` — does the pattern match? The simplest, most common use.

.match() `str.match(regex)`

Returns matches (or `null`). With `g`, an array of all matches; without, the first match plus capture groups.

.matchAll() `str.matchAll(regex)`

Iterator of all matches **with** their capture groups. Needs the `g` flag. **ES2020**.

.replace() / **.replaceAll()** `str.replace(regex, repl)`

Swap matches; `repl` can reference groups (`$1`) or be a function.

.split() `str.split(regex)`

Split on a pattern (e.g. `/\s+/` for any whitespace).

.exec() `regex.exec(str)`

Lower-level: returns the next match (stateful with `g`). Usually `matchAll` is clearer.

Pattern building blocks (the practical core)

\d \w \s character classes

Digit / word-character / whitespace. Capitalize to negate (`\D` = non-digit).

. **[abc]** **[^abc]** any / set / not-set

Any char / any one listed / any not listed. `[a-z]` is a range.

+ ***** **?** **{n,m}** quantifiers

One-or-more / zero-or-more / optional / a count range. Add `?` to make them lazy.

^ **\$** **\b** anchors

Start of string / end of string / word boundary.

() **(?<name>)** capture groups

Capture a matched piece for extraction; named groups read better. `(?:...)` groups without capturing.

a|b alternation

Match either side.

flags **g i m s u y d v** after the slash

`g` all, `i` ignore-case, `m` multiline, `s` dot-matches-newline, `u`/`v` unicode, `y` sticky, `d` indices.

Part 9 • Promise (static methods)

Book 1 covered how promises and `async`/`await` work. This part is the *toolbox*: the static `Promise.*` methods that combine several promises into one — the everyday tools for running async work in parallel.

Promise combinators CORE

IN PLAIN TERMS

A reminder: a **promise** represents a value that isn't ready yet (a network request in flight). Often you have *several* of these at once — three API calls, a batch of file reads — and you want to wait for them as a group rather than one-by-one. The `Promise.*` **combinators** are static methods (called on `Promise` itself) that take an array of promises and combine them into a single promise you can await.

`Promise.all` is the workhorse: run everything in parallel, get all results, fail if any one fails. `Promise.allSettled` is its tolerant sibling: wait for all and report each as success or failure. `Promise.race` and `Promise.any` resolve on the first to finish or first to *succeed*. Choosing the right one is mostly about how you want failures handled.

▼ KEY TERMS IN THIS SECTION

<code>===</code>	Strict equality operator — compares two values and returns <code>true</code> only if they're the same type and value, with no automatic conversion. Your default for comparing.
<code>==</code>	Loose equality operator — compares two values <i>after</i> converting them to a common type, which causes surprises. Avoid in favour of <code>===</code> .
<code>=></code>	Arrow — defines an <i>arrow function</i> , a short way to write a function (e.g. <code>x => x * 2</code>). The arrow separates the inputs from what the function does.
<code>**</code>	Exponentiation operator — raises a number to a power (<code>2 ** 3</code> is <code>8</code>).
<code>fulfilled</code>	Fulfilled — a promise that has succeeded and now holds a value.
<code>rejected</code>	Rejected — a promise that has failed and now holds an error.
<code>sibling</code>	Sibling — a node sharing the same parent as another node.
<code>static</code>	Static member — a property or method that belongs to the class itself, not to individual instances.

If async tasks don't depend on each other, `Promise.all` runs them together — awaiting them one-by-one instead is a common, invisible performance bug.

```
// run in PARALLEL and collect all results (fails fast if any reject):
const [user, posts, prefs] = await Promise.all([
  fetch("/api/user").then(r => r.json()),
  fetch("/api/posts").then(r => r.json()),
  fetch("/api/prefs").then(r => r.json()),
]); // all three requests fly at once — much faster than awaiting in sequence

// tolerate partial failure — get status for each:
const results = await Promise.allSettled(tasks);
results.forEach(r => {
  if (r.status === "fulfilled") use(r.value);
  else log(r.reason); // this one failed, others still ran
});

// first to SUCCEED wins (e.g. fastest mirror); ignores individual failures:
const fastest = await Promise.any([fetchA(), fetchB()]);

// first to SETTLE wins, success or failure (e.g. timeout pattern):
const out = await Promise.race([work(), timeout(5000)]);

// build a promise you resolve elsewhere (ES2024):
const {promise, resolve, reject} = Promise.withResolvers();
```

GOTCHA `Promise.all` rejects as soon as **any** promise rejects, and you lose the results of the ones that succeeded — use `Promise.allSettled` when you need every result regardless. `Promise.race` settles on the first to *finish whether it succeeded or failed*, so a fast rejection wins the race — that's different from `Promise.any`, which waits for the first *success*. Passing non-promise values is fine (they're treated as already-resolved).

WHEN TO USE These exist to express **parallelism** cleanly. The naive way to handle several async operations is to `await` them one after another — but if they're independent, that wastes time waiting for each to finish before starting the next. `Promise.all` starts them all at once and waits for the group, often turning three sequential 200ms requests into a single 200ms wait. The variants exist because real systems fail: `allSettled` when partial success is acceptable (load what you can), `any` for redundancy (first working source), `race` for timeouts (whichever finishes first). Knowing which combinator to reach for is a core backend/data skill — and it carries directly into the data-intensive API book later, where batching parallel work is central.

VERSION `Promise.all` / `race` are ES2015; `Promise.allSettled` is ES2020; `Promise.any` is ES2021; `Promise.withResolvers()` is ES2024; `Promise.try()` is ES2025.

Combinators & helpers

Promise.all() `Promise.all([...])`

Parallel; resolves with **all** results as an array, or rejects on the **first** failure. The default for independent tasks.

Promise.allSettled() `Promise.allSettled([...])`

Waits for all; resolves with a status object per promise (`fulfilled` / `rejected`). For tolerant batch work. ES2020.

Promise.race() `Promise.race([...])`

Settles as soon as the **first** promise settles — success *or* failure. Timeout patterns.

Promise.any() `Promise.any([...])`

Resolves with the first **success**; rejects only if all fail. Redundant sources. **ES2021**.

Promise.resolve() / **reject()** `Promise.resolve(v)`

Create an already-settled promise. Useful in adapters and tests.

Promise.withResolvers() `Promise.withResolvers()`

Returns `{promise, resolve, reject}` so you can settle it from outside. **ES2024**.

Promise.try() `Promise.try(fn)`

Runs `fn` and captures sync throws *and* async rejections uniformly as a promise. **ES2025**.

Part 10 • More built-ins

The remaining standard objects you’ll meet less often but should recognize: unique `Symbol` keys, the new iterator helper methods, the family of `Error` types, and binary data via typed arrays.

Symbol REFERENCE

IN PLAIN TERMS

A **Symbol** is a guaranteed-unique value. Every symbol you create is different from every other, even one made with the same description. Their main job is to serve as object keys that can never accidentally collide with another key — useful for adding "hidden" properties or for hooking into special language behaviors.

You create one with `Symbol("description")`. Because each is unique, they're used as non-colliding keys and to implement language protocols — most importantly `Symbol.iterator`, which is what makes an object work with `for...of` (you saw this in Book 1's iteration topic).

▼ KEY TERMS IN THIS SECTION

- ===** Strict equality operator — compares two values and returns `true` only if they're the same type and value, with no automatic conversion. Your default for comparing.
- ==** Loose equality operator — compares two values *after* converting them to a common type, which causes surprises. Avoid in favour of `===`.
- ...** Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.
- **** Exponentiation operator — raises a number to a power (`2 ** 3` is `8`).
- iterable** Iterable — any value you can step through one item at a time with `for...of` (arrays, strings, Maps, Sets).
- iterator** Iterator — an object that produces values one at a time via `.next()`; what `keys` / `values` / `entries` return.
- property** Property — a value on the live element object in JavaScript; may stay in sync with an attribute or diverge from it.
- spread** Spread — using `...` to expand a list or object into its individual pieces.

Symbols are mostly something you use via well-known ones like `Symbol.iterator` rather than create yourself — they're a building block library authors lean on more than app developers.

```
const id = Symbol("id");
const obj = {[id]: 123};           // a key that can't collide with any string key
obj[id];                          // 123

Symbol("x") === Symbol("x");     // false — every symbol is unique

// the well-known symbol you'll actually use — make an object iterable:
const range = {
  *[Symbol.iterator]() { yield 1; yield 2; yield 3; }
};
[...range];                      // [1, 2, 3]

Symbol.for("shared");            // global registry — same symbol across the app
```

WHEN TO USE `Symbol` exists to solve key collisions: if a library adds a property to your object using a string key, it might clobber yours (or vice versa); a symbol key is guaranteed unique, so it's safe for metadata and internal state. The *well-known symbols* (`Symbol.iterator`, `Symbol.asyncIterator`) exist so the language can let objects opt into behaviors — defining `[Symbol.iterator]` is literally how you make something work with `for...of` and spread. In everyday app code you'll mostly encounter symbols through these protocols rather than creating your own.

Symbol basics

Symbol() `Symbol(desc?)`

Creates a unique symbol. The description is just a label for debugging.

Symbol.iterator well-known symbol

Define this method to make an object iterable (`for...of`, spread). The key protocol hook.

Symbol.asyncIterator well-known symbol

Async version, for `for await...of` streams.

Symbol.for() / **keyFor()** global registry

Create/look up symbols shared across the whole program by a string key.

Iterator helpers

REFERENCE

IN PLAIN TERMS

Book 1 introduced iterables (things you can `for...of` over). **Iterator helpers** (new in ES2025) bring the familiar array methods — `map`, `filter`, `take` — directly to iterators, so you can process sequences *lazily*: one item at a time, computing only what you need, without first building a whole array. This is especially powerful for huge or infinite sequences.

Where `array.map().filter()` builds a new array at each step, iterator helpers form a **lazy pipeline**: nothing is computed until you pull values out, and `take(n)` can stop early — so they work on infinite generators and very large data without exhausting memory. You get them from any iterator (e.g. `array.values()`, a generator, `map.keys()`) or via `Iterator.from()`.

▼ KEY TERMS IN THIS SECTION

<code>===</code>	Strict equality operator — compares two values and returns <code>true</code> only if they're the same type and value, with no automatic conversion. Your default for comparing.
<code>==</code>	Loose equality operator — compares two values <i>after</i> converting them to a common type, which causes surprises. Avoid in favour of <code>===</code> .
<code>=></code>	Arrow — defines an <i>arrow function</i> , a short way to write a function (e.g. <code>x => x * 2</code>). The arrow separates the inputs from what the function does.
<code>**</code>	Exponentiation operator — raises a number to a power (<code>2 ** 3</code> is <code>8</code>).
<code>++</code>	Increment operator — adds 1 to a number variable (<code>i++</code>).
<code>generator</code>	Generator — a special function that can pause and resume, producing values on demand.
<code>iterable</code>	Iterable — any value you can step through one item at a time with <code>for...of</code> (arrays, strings, Maps, Sets).
<code>element</code>	Element — a node created from an HTML tag (a <code><div></code> , <code><p></code> , <code><button></code>); the most common kind of node.

The key difference from array methods: these are lazy. `take(3)` halts the whole pipeline after three results — which is why they can handle infinite sequences that would hang an array-based approach.

```
// lazy pipeline – no intermediate arrays, stops after 3 results:
const firstThreeEven = [1,2,3,4,5,6,7,8].values()
  .filter(n => n % 2 === 0)
  .map(n => n * 10)
  .take(3)
  .toArray();           // [20, 40, 60]

// works on an INFINITE generator because it's lazy:
function* naturals() { let n = 1; while (true) yield n++; }
const squares = naturals().map(n => n * n).take(5).toArray(); // [1,4,9,16,25]

Iterator.from(anyIterable); // wrap any iterable to get these helpers
```

WHEN TO USE Iterator helpers exist to bring array-style transformation to *streams* of data — sequences too large to hold in memory, or generated on demand, or genuinely infinite. With arrays, `map` and `filter` each build a full intermediate array, so you must have all the data up front and pay for every element. Iterator helpers instead pull one value through the whole chain at a time and can stop early, making them ideal for processing large files line-by-line, paginated API results, or generated sequences. This connects directly to the **data-intensive** work later in the series, where streaming rather than buffering is the whole game.

VERSION Iterator helpers (`map`, `filter`, `take`, `drop`, `flatMap`, `reduce`, `some`, `find`, `every`, `toArray`, and `Iterator.from`) are ES2025 — verify runtime support before relying on them in production.

Helper methods (on any iterator)

`.map()` / `.filter()` `iterator.map(fn)`

Lazy transform / keep — like the array versions but one item at a time.

`.take()` / `.drop()` `iterator.take(n)`

Take the first `n` / skip the first `n`. `take` stops the pipeline early.

`.flatMap()` `iterator.flatMap(fn)`

Map then flatten, lazily.

`.reduce()` / `.some()` / `.find()` / `.every()` terminators

Consume the iterator to a single result.

`.toArray()` `iterator.toArray()`

Pull the lazy pipeline into a real array (the usual final step).

`Iterator.from()` `Iterator.from(iterable)`

Wrap any iterable so the helper methods are available.

Error types CORE

IN PLAIN TERMS

Book 1 covered `try` / `catch` / `throw`. This is the family of built-in `Error` objects you actually throw and catch.

`Error` is the base; specific types like `TypeError` and `RangeError` describe *what kind* of thing went wrong, so a `catch` block can react differently to different failures.

You'll mostly throw the generic `Error` with a clear message, and create custom subclasses for your app's own failure types. The built-in subclasses (`TypeError`, `RangeError`, `SyntaxError`) are thrown automatically by the language and are also useful to throw yourself for precise meaning. Every error carries `message`, `name`, and a `stack` trace.

▼ KEY TERMS IN THIS SECTION

**	Exponentiation operator — raises a number to a power (<code>2 ** 3</code> is <code>8</code>).
constructor	Constructor — the special method that sets up a new instance when you use <code>new</code> .
property	Property — a value on the live element object in JavaScript; may stay in sync with an attribute or diverge from it.
promise	Promise — an object standing in for a value that isn't ready yet; it later succeeds or fails.
syntax	Syntax — the grammar rules for how code must be written so the computer can understand it.
parse	Parse — read text and turn it into structured data (e.g. <code>JSON.parse</code>).
TDZ	TDZ (temporal dead zone) — the period before a <code>let</code> / <code>const</code> is initialized, where using it throws an error.

Branch on error type with `instanceof` to react appropriately, and re-throw anything you didn't mean to handle. Custom subclasses let you carry structured context (which field, which record).

```
throw new Error("Something went wrong");           // the generic case
throw new RangeError("Page must be 1 or more");    // a specific, meaningful type

// a custom error type for your app, with extra context:
class ValidationError extends Error {
  constructor(field, message) {
    super(message);
    this.name = "ValidationError";
    this.field = field;           // attach data the caller can use
  }
}

try {
  doThing();
} catch (err) {
  if (err instanceof ValidationError) highlightField(err.field);
  else if (err instanceof TypeError) log("bad type:", err.message);
  else throw err;               // re-throw what you can't handle
}

// chain an underlying cause (ES2022):
throw new Error("Failed to load", {cause: originalError});
```

WHEN TO USE The error types exist so failures can be told apart programmatically. If everything were a generic `Error`, a `catch` block could only inspect message strings (fragile) to decide what happened. With distinct types, `catch (e) { if (e instanceof TypeError) ... }` reacts precisely, and custom subclasses (`ValidationError`, `NotFoundError`) let your code attach and read structured context. This matters enormously in backend/API work later: an API turns a `ValidationError` into a 400 response and a `NotFoundError` into a 404, just by checking the error's type. Always throw `Error` objects (never bare strings) so the `stack` trace and `name` survive.

VERSION The error `cause` option (`new Error(msg, {cause: e})`) is ES2022; `AggregateError` (carrying multiple errors, used by `Promise.any`) is ES2021.

The Error family

Error `new Error(message, {cause?: e})`

The base type. Carries `message`, `name`, `stack`. Throw this for general failures; subclass it for your own.

TypeError `thrown` / `new TypeError()`

Wrong type or operation (calling a non-function, reading a property of `null`). Thrown by the language and useful to throw yourself.

RangeError `new RangeError()`

A value outside its allowed range (bad array length, out-of-range number).

SyntaxError `thrown`

Invalid syntax — thrown by `JSON.parse` on bad input, and by the engine on malformed code.

ReferenceError `thrown`

Using a name that doesn't exist (often a typo or a TDZ access).

AggregateError `new AggregateError(errors)`

Wraps several errors in one — what `Promise.any` rejects with. ES2021.

custom extends Error `class X extends Error`

Your own error types — set `this.name` and attach context fields. The basis for typed error handling.

Binary data REFERENCE

IN PLAIN TERMS

Most of the time you work with numbers, strings, and objects. But sometimes you handle **raw binary data** — bytes from a file, an image, a network stream, audio. **Typed arrays** (like `Uint8Array`) are array-like views over raw memory (an `ArrayBuffer`) that let you read and write those bytes efficiently. This is a specialized corner you'll meet mainly when dealing with files, streams, or performance-critical data.

An `ArrayBuffer` is a fixed block of raw bytes; a typed array (`Uint8Array`, `Float64Array`, etc.) is a typed window onto it that you index like a normal array. You'll encounter them when reading files (`await blob.arrayBuffer()`), handling binary network data, or working with the data-intensive streaming covered later in the series.

▼ KEY TERMS IN THIS SECTION

****** Exponentiation operator — raises a number to a power (`2 ** 3` is `8`).

array-like Array-like — an object with a `length` and numbered keys but no array methods (e.g. a DOM `NodeList`); convert with `Array.from`.

index Index — an item's numbered position in an array or string, starting at 0.

`TextEncoder` / `TextDecoder` *convert between strings and raw bytes (UTF-8) — the bridge you'll use most when binary data is really text.*

```
const buffer = new ArrayBuffer(8);    // 8 raw bytes
const bytes = new Uint8Array(buffer); // view them as unsigned 8-bit ints
bytes[0] = 255;
bytes.length;                        // 8

// from a normal array, and back:
const u8 = Uint8Array.from([1, 2, 3]);
Array.from(u8);                      // [1, 2, 3]

// where you actually meet them:
const data = new Uint8Array(await blob.arrayBuffer()); // file bytes
const text = new TextDecoder().decode(data);           // bytes → string
const back = new TextEncoder().encode("hi");           // string → bytes
```

WHEN TO USE Typed arrays exist because JavaScript's normal arrays are flexible objects, not efficient byte storage — fine for app data, wrong for megabytes of image pixels or audio samples. Typed arrays give you compact, fixed-type, memory-efficient storage that maps directly onto how files, network protocols, and hardware represent data. **You'll meet them** when reading uploaded files, processing streams, decoding binary formats, or squeezing performance out of large numeric datasets — which is exactly the territory of the data-intensive API and automation books later. For everyday work, you rarely need them; recognize them and know they're the tool when raw bytes appear.

Binary building blocks

ArrayBuffer new ArrayBuffer(bytes)

A fixed-length block of raw memory. You don't read it directly — you put a view over it.

Uint8Array / Int32Array / Float64Array / ... typed arrays

Typed, indexable views over a buffer. `Uint8Array` (bytes) is the most common.

DataView new DataView(buffer)

Read/write mixed types at byte offsets with explicit endianness — for parsing binary formats.

TextEncoder / TextDecoder encode / decode

Convert between strings and UTF-8 bytes. The usual bridge when binary data is text.

Blob / File web objects

Browser file/binary objects; `.arrayBuffer()` / `.text()` extract their contents. (Covered more in later books.)

API Index

Every operator, keyword, and statement form in one place — the quick lookup for “what does this symbol mean?”

TOKEN	KIND	MEANING
<code>[]</code>	Array literal	Create an array: the everyday way.
<code>Array.from()</code>	Array static	Build an array from an iterable or array-like (optional map fn).
<code>Array.of()</code>	Array static	Array from arguments; avoids the new Array(n) empty-slots trap.
<code>Array.fromAsync()</code>	Array static	Build an array from an async iterable. Returns a promise. ES2025.
<code>Array.isArray()</code>	Array static	Reliable test for whether a value is an array.
<code>.push()</code> / <code>.pop()</code>	Array method	Add to / remove from the END. Mutates.
<code>.shift()</code> / <code>.unshift()</code>	Array method	Remove from / add to the START. Mutates.
<code>.splice()</code>	Array method	Insert/remove at any position. Mutates; returns removed items.
<code>.slice()</code>	Array method	Shallow copy of a portion. Does not mutate.
<code>.concat()</code>	Array method	New array joining this with others. Non-mutating.
<code>.sort()</code>	Array method	Sort in place; pass a comparator for numbers. Mutates.
<code>.reverse()</code>	Array method	Reverse in place. Mutates.
<code>.find()</code> / <code>.findIndex()</code>	Array method	First item / index matching a test.
<code>.findLast()</code> / <code>.findLastIndex()</code>	Array method	Search from the end. ES2023.
<code>.indexOf()</code> / <code>.lastIndexOf()</code>	Array method	Position of an exact value, or -1.
<code>.includes()</code>	Array method	True if an exact value is present (finds NaN).
<code>.some()</code> / <code>.every()</code>	Array method	Does any / do all items pass a test (short-circuits).
<code>.at()</code>	Array method	Item at an index; supports negatives. ES2022.
<code>.map()</code>	Array method	New array with a function applied to each item.
<code>.filter()</code>	Array method	New array of items passing a test.
<code>.reduce()</code>	Array method	Collapse an array to a single value (pass an initial).

TOKEN	KIND	MEANING
<code>.flatMap()</code> / <code>.flat()</code>	Array method	Map-then-flatten / flatten nested arrays. ES2019.
<code>.forEach()</code>	Array method	Run a function per item for side-effects. Can't break/await.
<code>.join()</code>	Array method	Join items into a string with a separator.
<code>.entries()</code> / <code>.keys()</code> / <code>.values()</code>	Array method	Iterators of [i,v] / indexes / values.
<code>.with()</code>	Array method	Copy with one index replaced. Immutable. ES2023.
<code>.toSorted()</code> / <code>.toReversed()</code> / <code>.toSpliced()</code>	Array method	Immutable copies of sort/reverse/splice. ES2023.
<code>structuredClone()</code>	Global	Deep copy of arrays/objects, including nested data.
<code>.length</code>	String	Count of UTF-16 code units.
<code>.at()</code> / <code>[i]</code> / <code>.charAt()</code>	String	Character at an index (.at supports negatives).
<code>.includes()</code>	String	True if it contains a substring. Case-sensitive.
<code>.startsWith()</code> / <code>.endsWith()</code>	String	Prefix / suffix test.
<code>.indexOf()</code> / <code>.lastIndexOf()</code>	String	Position of a substring, or -1.
<code>.slice()</code>	String	Extract a section; supports negative indexes. Preferred.
<code>.substring()</code> / <code>.substr()</code>	String	Older extractors; prefer slice (substr is deprecated).
<code>.trim()</code> / <code>.trimStart()</code> / <code>.trimEnd()</code>	String	Remove surrounding whitespace. ES2019 for the sided forms.
<code>.toLowerCase()</code> / <code>.toUpperCase()</code>	String	Change case for compare/display.
<code>.replace()</code>	String	Replace the first match (or all with a /g regex).
<code>.replaceAll()</code>	String	Replace every occurrence. ES2021.
<code>.padStart()</code> / <code>.padEnd()</code>	String	Pad to a target width. ES2017.
<code>.repeat()</code>	String	Repeat the string n times.
<code>.split()</code>	String	Break into an array on a separator or regex.
<code>.normalize()</code>	String	Normalize Unicode so accents compare equal.
<code>Number()</code>	Conversion	Strict text-to-number; whole string must be numeric.
<code>parseInt()</code> / <code>parseFloat()</code>	Global	Lenient leading number from text (pass radix 10 to parseInt).
<code>.toFixed()</code>	Number	Fixed decimal places, as a string. For display.
<code>.toString(radix)</code>	Number	Number to text in a given base (2, 16, ...).
<code>.toLocaleString()</code>	Number	Locale-formatted number. See Intl for control.
<code>Number.isNaN()</code> / <code>isFinite()</code>	Number static	Correct NaN / finite checks (no coercion).

TOKEN	KIND	MEANING
<code>Number.isInteger()</code> <code>/ isSafeInteger()</code>	Number static	Whole-number / safe-range checks.
<code>Number.MAX_SAFE_INTEGER</code> / <code>EPSILON</code>	Number static	Largest exact integer / float comparison tolerance.
<code>Math.round()</code> / <code>floor()</code> / <code>ceil()</code> / <code>trunc()</code>	Math	Integer rounding (down, up, toward zero).
<code>Math.max()</code> / <code>min()</code>	Math	Largest / smallest of arguments (spread an array).
<code>Math.abs()</code> / <code>sign()</code>	Math	Magnitude / sign of a number.
<code>Math.pow()</code> / <code>sqrt()</code>	Math	Power / square root (** operator also raises to a power).
<code>Math.random()</code>	Math	Random float in [0,1). <code>floor(random()*n)</code> for integers.
<code>Object.keys()</code> / <code>values()</code> / <code>entries()</code>	Object static	Own keys / values / [k,v] pairs as arrays.
<code>Object.fromEntries()</code>	Object static	Build an object from [k,v] pairs. ES2019.
<code>Object.assign()</code>	Object static	Copy props into a target (mutates it). Shallow merge.
<code>{...spread}</code>	Object literal	Copy/merge into a new object; later keys win. ES2018.
<code>Object.groupBy()</code>	Object static	Group items into an object by a key function. ES2024.
<code>Object.freeze()</code> / <code>isFrozen()</code>	Object static	Make read-only (shallow) / test it.
<code>Object.hasOwn()</code>	Object static	Safe own-property check. Replaces <code>hasOwnProperty</code> . ES2022.
<code>Object.create()</code> / <code>getPrototypeOf()</code>	Object static	Prototype-level creation / inspection.
<code>Object.defineProperty()</code>	Object static	Fine-grained property control. For libraries.
<code>new Map()</code>	Map	Key-value store with any key type, insertion order, <code>.size</code> .
<code>map.set()</code> / <code>get()</code> <code>/ has()</code> / <code>delete()</code>	Map	Add-or-update / read / check / remove.
<code>map.size</code>	Map	Number of pairs.
<code>new Set()</code>	Set	Collection of unique values.
<code>set.add()</code> / <code>has()</code> <code>/ delete()</code>	Set	Add / fast membership test / remove.
<code>[...new Set(arr)]</code>	Idiom	De-duplicate an array.
<code>set.union()</code> / <code>intersection()</code> / <code>difference()</code>	Set	Set algebra. ES2025.
<code>new WeakMap()</code> / <code>WeakSet()</code>	Weak collection	Weakly-held object keys; auto-cleaned. No size/iteration.
<code>JSON.stringify()</code>	JSON	Data to JSON text. Third arg = indentation.
<code>JSON.parse()</code>	JSON	JSON text to data. Wrap in try/catch for untrusted input.
<code>new Date()</code>	Date	Now, or a moment from ms / ISO string / parts (month 0-based).
<code>Date.now()</code>	Date static	Current time as ms since epoch. For timing.

TOKEN	KIND	MEANING
<code>.getTime()</code>	Date	Ms since epoch; subtract two dates for a duration.
<code>.getFullYear() / getMonth() / getDate()</code>	Date	Year / month (0-11) / day-of-month.
<code>.toISOString()</code>	Date	Sortable, storable UTC string.
<code>Intl.NumberFormat</code>	Intl	Locale number/currency/percent formatting.
<code>Intl.DateTimeFormat</code>	Intl	Locale date/time formatting.
<code>Intl.RelativeTimeFormat / ListFormat</code>	Intl	"3 days ago" / "A, B, and C".
<code>/pattern/flags</code>	RegExp literal	A text pattern; flags <code>g i m s u y d v</code> .
<code>regex.test()</code>	RegExp	Does the pattern match? true/false.
<code>.match() / .matchAll()</code>	String+regex	Extract matches (matchAll needs <code>/g</code>). ES2020 for matchAll.
<code>.replace() / .replaceAll()</code>	String+regex	Swap matches; <code>\$1</code> references capture groups.
<code>(?<name>...)</code>	RegExp	Named capture group. ES2018.
<code>Promise.all()</code>	Promise static	Parallel; all results, or reject on first failure.
<code>Promise.allSettled()</code>	Promise static	Wait for all; status per promise. ES2020.
<code>Promise.race() / any()</code>	Promise static	First to settle / first to succeed. any is ES2021.
<code>Promise.resolve() / reject()</code>	Promise static	Create an already-settled promise.
<code>Promise.withResolvers()</code>	Promise static	{promise, resolve, reject}. ES2024.
<code>Symbol()</code>	Symbol	Unique value, mainly for non-colliding keys.
<code>Symbol.iterator</code>	Symbol	Define to make an object iterable (for...of, spread).
<code>Iterator helpers</code>	Iterator	<code>.map/.filter/.take/.drop/.toArray</code> — lazy. ES2025.
<code>Error / TypeError / RangeError</code>	Error	Base and specific error types; branch with instanceof.
<code>class X extends Error</code>	Error	Custom error types with structured context.
<code>ArrayBuffer / Uint8Array</code>	Binary	Raw bytes / typed view over them.
<code>TextEncoder / TextDecoder</code>	Binary	Convert between strings and UTF-8 bytes.